딥러닝을 활용한
독버섯 판단 및 서비스 구현

Deep Learning Based Identification
of Poisonous Mushrooms and Service Implementation

배고픈 GPT

과목명	딥러닝프로젝트 어드벤쳐 KCC	지도교수	양영욱
팀 구성원	학번	성명	학과
	20251928	이창규	컴퓨터공학과
	20251932	황재민	컴퓨터공학과
	20221317	지수정	컴퓨터공학과
	20221363	전준현	컴퓨터공학과

국립강릉원주대학교

Index

Abstract **1**

1. Introduction **1**

2. Related Works **2**

2.1. Study of Classifying mushroom images using CNN-based models 2

2.2. A Study on the Implementation of a Poisonous Mushroom Determination Program through Deep Learning ... 3

2.3. Picture Mushroom – Identifies Mushrooms 4

2.4. Mushroom Information Sharing Community 4

3. Construction of the Dataset **5**

3.1. Training Data Construction 5

3.2. Data Augmentation 6

4. Model Experiments and Analysis **7**

4.1. Model Architecture 7

4.1.1. AlexNet 7

4.1.2. VGGNet 8

4.1.3. GoogLeNet v4 8

4.1.4. ResNet-50 9

4.2. Model-Specific Training Results 10

4.2.1. AlexNet 10

4.2.2. VGGNet 11

4.2.3. GoogLeNet v4 12

4.2.4. ResNet-50 13

4.3. Result Analysis 15

4.4. Final Training of the Model Deployed in the Service 15

5. Design and Implementation of the Service Program **17**

5.1. Service Architecture 17

5.2. Methodology for the Integration of Deep Learning Models into Web Page 19

6. Conclusions **19**

7. References **20**

Appendix A: Mid-Term Q&A **22**

Appendix B: Final-Term Q&A **23**

Appendix C: Changes in Cost and Training Accuracy by Epoch **24**

Appendix D: Webpage Design and Page Components **25**

Appendix E: Team Member Introduction, Roles, and Contributions **34**

Deep Learning Based Identification of Poisonous Mushrooms and Service Implementation

Jaemin Hwang[1], Changgyu Lee[1], Junhyeon Jeon[1], Sujeong Ji[1]

[1]*Dept. of Computer Science and Engineering, Gangneung-Wonju National University, Gangwon-do, 26403, Korea*

Abstract

Every autumn, various government agencies – including the Rural Development Administration and the Korea Forest Service – issue warnings to alert the public to the frequent incidence of poisoning accidents caused by wild mushrooms. In this study, we hypothesize that such accidents persist because individuals can not reliably determine whether a discovered mushroom is poisonous. To address this gap and prevent future incidents, we designed and implemented a service capable of identifying toxic species in real time.

We obtained photographs and descriptive data on both poisonous and edible wild mushrooms from the Rural Development Administration. Focusing on deep learning classification architectures, we evaluated and compared the predictive performance of several published models. Based on these experiments, we selected ResNet-50 – the model that achieved the highest accuracy – and trained it on our assembled data-set. Finally, we deployed this trained model within a web application, creating a user-friendly service that can be accessed in everyday settings to assist in safe mushroom foraging.

Keywords: *CNN, Classification, Poisonous Mushroom, Web Service*

딥러닝을 활용한 독버섯 판단 연구 및 서비스 구현

황재민[1], 이창규[1], 전준현[1], 지수정[1]

[1]강릉원주대학교 컴퓨터공학과

요 약

매년 가을이 되면 농촌진흥청, 산림청 등의 다양한 정부 기관에서 야생 버섯으로 인한 중독사고가 빈번하게 발생하기에 주의해야 함을 안내하고 있다. 본 연구에서는 이러한 안내에도 불구하고 버섯을 발견하였을 때 독버섯인지에 대한 정보를 정확하게 얻을 수 없어 중독사고가 일어난다고 생각하여, 사고 예방을 위하여 문제를 해결할 수 있는 서비스를 구현하여 제공하고자 한다.

정부 기관인 농촌진흥청에서 소개하고 있는 독버섯과 야생 버섯에 대한 사진과 정보를 학습에 사용한다. 딥러닝 모델 중, 분류 모델을 중점으로 발표된 다양한 모델을 사용하여 예측 결과를 비교하였다. 도출된 결과를 기반으로, 최종적으로 가장 높은 정확도를 보인 모델인 ResNet-50을 사용하여 구축한 데이터 세트를 학습시켰다. 이를 이용하여 웹페이지 구축을 통해 실생활에서도 사용할 수 있는 서비스를 구축하였다.

Keywords: *CNN, 분류, 독버섯, 웹 서비스*

---

1. Introduction

매년 가을이 되면 뉴스 또는 다양한 매체에서 산행 사고를 주의하는 내용과 함께 독버섯에 관한 경고를 하곤 한다. 이유 중 하나로는 야산에서 임산물을 채취하는 모습을 쉽게 발견할 수 있는데, 임산물 중 독버섯을 식용버섯으로 오인하여 섭취함으로써 사고가 빈번하게 일어나기 때문이다. 또 다른 이유로는 겨울이 끝나고 날씨가 따뜻해지는 시기에는 야외 활동과 산행의 빈도가 증가하면서, 이전의 이유와 같은 원인을 일으키기도 한다. 즉, 시기만 다를 뿐 지속해서 사건과 사고가 빈번하게 발생하기에 2024년에도 뉴스 매체에서 소방장이나 국립산림과학원 종사자와의 인터뷰 장면을 방송하여 심각성을 알렸다[1][2].

국립산림과학원 산림 미생물 이용 연구과의 이경태 연구사는 “가을철 가을 단풍을 즐기기 위하여 가족들이나 친지들과 함께 산행하러 갔다가 사고로 이어질 수 있는 위험이 있습니다.”라고 말하며, 순간의 선택이 사고로 이어짐을 강조하였다. 산림청 국립수목원 국가표준 버섯목록에 따르면 630속 2,221종 중에서 약 77%가 식용 불명 및 독버섯이라 설명하였다. 또한, 외형적으로 매우 유사한 형태를 가지고 있어 일반인들이 구별하기는 쉽지 않다고 이야기했다.

농촌진흥청에서는 식품의약품안전처 조사를 다음과 같이 분석하였다[3]. 최근 10년간(2014-2023) 야생 버섯으로 인한 중독사고는 5건, 이에 따라 발생한 환자 수는 38명이라 밝혔다. 1건당 평균 환자 수가 7.6 명인 것을 보면 야생 버섯을 가족, 지인과 나눠 먹어 피해가 확산한 것으로 해석할 수 있다고 설명한다. 또한, 버섯과 관련된 잘못된 민간 속설의 예를 이야기하며, 잘못된 정보를 바로잡는 자료도 함께 제공하였다.

산림청에서는 2023년 보도자료를 통해 버섯 2,170종 중 먹을 수 있는 것은 493종뿐이기에, 독버섯을 주의해야 함을 설명했다[4]. 또한, 버섯은 건강에 좋은 식재료로 알려져 비전문가들도 온라인에서 얻은 불분명한 지식이나 일반 도감을 활용해 야생 버섯을 채취하여 사고가 발생함을 안내하며, 야외에서 버섯을 채취해 먹는 행위는 사망 사고와 같은 대형 사고로 이어질 수 있기에 자제해야 한다고 주의를 당부하였다.

독버섯 식용으로 인해 발생하는 사고를 예방하기 위한 가장 좋은 방법은 정부 기관에서 보도자료를 통해 설명하는 바와 같이 야외에서 버섯을 함부로 채취하지 않는 것이 가장 바람직하다. 이유 중 하나로는 버섯을 채취하였을 때 해당 버섯이 어떠한 버섯인지를 구별하는 데 상당한 어려움이 따르기 때문이다. 구별하는 방법으로는 전문가의 지식에 의존하거나, 정부 기관에서 제공하는 도감에 수록된 사진과 설명을 비교하는 것이 있다. 또는, 인터넷에 공개된 정보만을 가지고 판단해야 한다는 제한성이 있기에, 전문가가 아닌 경우에는 오인할 가능성이 농후하다.

버섯을 판단하는 문제를 해결하고자 할 때는 위에서 설명한 바와 같이 상당히 제한된 환경 속에서 판단해야 한다. 이러한 제한된 환경으로 인하여 채취한 야생 버섯을 잘못 오인하게 되고, 이를 섭취함으로써 사고를 유발한다고 생각하였기에 제시한 문제를 해결하는 데 조금이나마 이바지하고자 연구를 진행하고자 한다. 또한, 연구를 기반으로 새롭게 구축한 모델을 기반으로 하여 해당 모델이 사용자가 사용할 수 있도록 하는 서비스를 구축하여 제공함으로써, 연구에서 그치지 않고 실생활에서 사용되도록 제공하고자 한다.

2. Related Works

2.1. Study of Classifying mushroom images using CNN-based models

해당 연구는 본 연구에서 해결하고자 하는 문제와 비슷한 문제를 해결하기 위한 초기 연구를 진행하였기에 소개하고자 한다[5]. 해당 연구는 Kaggle에서 제공하고 있는 버섯 이미지 데이터 세트[6]를 사용하였다. 데이터 세트 안에 있는 이미지를 정확도 향상을 위하여 전처리 단계로 이미지 크기 조정과 버섯이 이미지 내에서 중앙에 위치할 수 있도록 조절하였다. 또한, 이미지 정규화를 진행하는 단계를 거쳤다고 설명한다.

학습에 이용한 모델은 총 4개로 LeNet[7], ResNet50[8], ResNet-152, MobileNet[9]을 사용하였다. 결과를 나타내는 과정에서는 전처리 과정을 거치기 전과 거친 이후로 나누어 정확도를 나타내어 분석한 결과를 소개하였다. LeNet의 경우에는 구조가 단순하기에 학습 정확도에서 큰 차이를 보이지는 않았으나, ResNet이나

MobileNet의 경우에는 3~5%의 향상된 결과를 보였다고 밝혔다. 이를 통하여 데이터 전처리 과정이 중요함을 설명하였으며, 75%의 정확도를 보였음을 설명한다.

더 나아가서 Validation Set을 이용하여 도출한 학습 정확도와 Test Set을 이용하여 도출한 학습 정확도의 차이가 작은 것을 근거로 새로운 데이터에 대해서도 일반화가 잘 된다는 결론으로 마무리하였다.

해당 연구에서 한계점으로 제시한 바에 따르면, 북유럽에서 흔한 버섯뿐 아니라 국내에서 흔히 볼 수 있는 버섯 데이터들로 추가 학습이 이루어진다면 실생활에 적용이 가능한 모델로 활용할 수 있다고 이야기한다. 그러나 해당 연구에서는 단순하게 독버섯과 식용버섯을 분류하고, 모델이 독버섯을 판단하는 정확성이 어느 정도인지, 그리고 데이터의 전처리 과정의 중요성에 대해 언급하고 있다. 이는 연구의 초기 접근 과정에서 독버섯 식용으로 인한 사건과 사고의 수를 소개하였으나, 해당 연구를 국내에서 사용될 수 있도록 적용하기에는 상당히 어려우며, 한계점이 명확하다고 판단하였다.

본 연구에서는 해당 한계점을 극복하여 실생활에서 사용할 수 있도록 국내에서 채취하거나 발견할 수 있는 식용버섯과 독버섯 이미지로 구성된 데이터 세트를 구축하여 학습에 이용되도록 하고자 한다. 이를 통하여 국내에서 본 연구를 통해 제공하고자 하는 서비스를 이용하여, 해당 연구나 본 연구에서 문제로 제시한 독버섯 중독으로 인한 사건과 사고를 예방하는 데 조금이나마 이바지하고자 한다.

2.2. A Study on the Implementation of a Poisonous Mushroom Determination Program through Deep Learning

해당 연구는 본 연구에서 해결하고자 하는 문제와 같은 문제를 해결하고자 하였으나, 문제에 접근하는 방식에서 차이가 있다[10]. 2.1.을 통해 소개한 학술지의 연구처럼 Kaggle에서 제공하고 있는 데이터인 이미지로 이루어져 있는 데이터 세트를 사용하였다. 또한, 가장 높은 학습 정확도를 보이는 모델을 찾기 위하여 LeNet, ResNet-50, ResNet-152, MobileNet 모델을 학습에 사용하였다. 초기 연구를 통하여 ResNet과 MobileNet 모델에서 3~5% 향상된 결과를 보였다. 초기 연구에서 성과를 보였기에, 해당 연구에서는 더욱 높은 학습 정확도를 얻기 위하여 Open Source로 공개된 코드 중에서 Activation Function, Optimizer를 수정하며 학습 정확도를 높이는 것을 목적으로 한다고 설명하였다.

해당 연구에서 문제를 해결하기 위한 세부 과정은 다음과 같다. Open Source에서의 Activation Function은 ReLU[11]를 사용하였으나, 과적합 문제를 해결하기 위하여 해당 연구에서는 2가지의 Activation Function을 변경하며 학습에 이용하였고, 연구 결과를 토대로, 최종적으로 ELU[12]를 사용하였다. Optimizer는 Open Source의 경우에는 Adam[13]을 사용하였으나, 해당 연구에서는 배치의 개념을 적용하지 않았기 때문에 이를 배제하면서 더욱 완만한 학습 곡선을 그릴 수 있는 RMSprop[14]을 채택하여 사용하였다.

초기 연구에서는 기존의 Open Source에서 제공한 데이터 세트를 사용하였으나, 독버섯 판정 프로그램을 만들어 문제를 해결하고자 함을 기반으로 하여 실질적인 성과를 얻고자 데이터 세트를 새롭게 구축하였다고 소개한다. 해당 연구에서는 식용버섯으로는 흰주름버섯, 마귀광대버섯, 노란달걀버섯을, 독버섯으로는 독우산광대버섯, 붉은점박이광대버섯, 개나리광대버섯을 선정하여 각 버섯 당 100장의 이미지를 확보하였다. 또한, 정확한 분석을 위하여 1:1 정방향 비율로 재조정하고, 이미지 가운데에 버섯이 위치할 수 있도록 수정한 후 저장하였다고 설명한다.

이러한 과정을 거쳐 학습을 진행하였으며, 학습 정확도는 79%, 69%, 74%를 나타내었다. 또한, 학습에 사용하지 않은 이미지 3장을 가지고 최종적으로 예측하도록 하였는데, 2개의 이미지는 정답을 나타내었고, 1개의 이미지는 정답을 나타내지 못하였다.

해당 연구에서도 한계점으로는 버섯의 이미지를 수집하는 데 상당한 어려움을 겪었고, 이에 따라 높은 학습 정확도를 얻지 못하였다고 나타내었다. 본 연구에서는 이러한 문제점을 해결하고, 해당 연구에서 변형한 접근 방식을 기존의 접근 방식을 사용하여 학습을 진행하고자 한다. 즉, 공개된 이미지를 더 많이 수집하고, 데이터 세트의 크기를 키우고자 데이터 증강 기법을 사용함으로써 해당 연구에서 발생한 한계점을 해결하고자

한다. 또한, 데이터 세트가 커짐에 따라 배치의 개념이 적용되기 때문에 해당 연구에서는 Optimizer로 RMSprop을 사용하였으나, 본 연구에서는 Adam을 사용하고자 한다.

2.3. Picture Mushroom – Identifies Mushrooms

2.1.과 2.2.에서는 버섯을 분류하는 것에 관한 연구를 중점적으로 살펴보았다면, 2.3.과 2.4.에서는 이를 응용하여 어떠한 형태로 사용자에게 서비스를 제공하고 있는지를 살펴보고자 한다.

'Picture Mushroom – Identifies Mushrooms'라는 서비스는 Google Play에서 Next Vision Limited가 제공하는 애플리케이션으로, 모든 버섯을 식별하고, 버섯을 식별하는 가이드를 제공하며, 나만의 버섯 수집기를 만들 수 있는 기능을 제공한다고 소개하고 있다[15]. 해당 서비스에서 제공하고 있는 기능 중 가장 중요한 버섯을 사진을 통해 식별하고, 어떠한 버섯인지를 나타낸다는 점에서 구현하고자 하는 기능을 담은 서비스를 구현하고 제공하고 있다고 판단할 수 있다. 또한, 버섯을 식별하였을 때 해당 버섯이 독버섯일 경우에 '치명적'이라는 문구를 보임으로써 사용자에게 경고한다는 점에서 구현하고자 하는 기능과 유사한 기능을 제공하고 있음을 확인할 수 있다. 차이점으로는 해당 서비스는 사용자가 촬영하여 판단을 요구한 버섯의 이미지를 애플리케이션 내에 저장할 수 있도록 함으로써, 이전에 어떠한 버섯을 촬영하였는지 모아서 볼 수 있도록 제공하고 있다. 또한, Google 검색과 연동하여, 궁금한 버섯에 관련된 정보를 얻을 수 있도록 제공하고 있다.

하지만, 단점으로는 구독이라는 시스템으로, 해당 서비스를 사용자에게 무료로 제공하는 것이 아닌 일정 금액을 받고 제공하고 있다는 점이다. 또한, Google Play에서 제기되고 있는 문제는 사용자가 해당 서비스를 사용하기 위하여 애플리케이션을 내려받고 사용하는 시점에서 구독 시스템에 관하여 명확한 설명은 하지 않고 무료체험 기간인 7일 이후 자동으로 가입이 되도록 하고 있으며, 구독 취소와 관련된 내용은 자세하게 설명하지 않고 있다는 것도 문제점으로 지적하고 있다. 또한, 해당 문제를 해결하기 위한 소통 창구가 마련되지 않아 사용자들이 더욱 어려움을 겪게 된다는 것도 문제가 될 수 있다고 파악하였다. 제기된 또 다른 문제점으로는 버섯의 한국 명칭이 나타나지 않는 것, 정확도에 문제가 있다는 것이지만, 사용자 대부분이 구독을 취소해달라는 것을 요청하고 있음을 확인할 수 있다.

해당 서비스도 사용자가 어떠한 버섯인지를 판단하는 데 도움을 얻을 수 있는 하나의 수단으로 선택될 수 있다. 간편하게 버섯의 이미지만 있으면, 해당 버섯이 어떠한 버섯인지를 알려주고, 해당 버섯에 관련된 정보를 확인할 수 있도록 한다는 것. 더 나아가서, 지금까지 어떠한 버섯을 자신이 판단을 요청했는지를 한 눈에 볼 수 있도록 제공하고 있다는 점은 큰 장점이며, 이를 제공하고자 하는 서비스에 적용하여 제공하면 사용자들에게 더욱 편리한 서비스가 될 수 있을 것으로 판단한다. 하지만, 살펴본 해당 서비스에서 제기된 문제점처럼 구독 서비스를 통하여 수익화하는 방법으로는 더욱 신중한 접근이 필요하다는 것을 깨달을 수 있었다. 만약, 수익화 구조로 서비스를 제공한다면, 수익에 합당한 더욱 정교한 서비스를 제공해야 하며, 사용자들의 의견에 적극적으로 응하는 것 또한 필요함을 확인할 수 있었기에, 해당 서비스를 구현할 때에는 이러한 문제점은 발견되지 않도록 구현하여 서비스를 제공하고자 한다.

2.4. Mushroom Information Sharing Community

알고자 하는 어떠한 정보를 얻기 위해서는 책에 기록된 내용이나 전문가의 도움을 받거나 할 수 있다. 그러나, 정보화 시대인 지금은 인터넷상에 궁금한 내용을 입력하여 검색한 이후에 답을 찾아 나가는 것이 시간과 공간에 제약을 받지 않을 수 있다는 것과 답을 얻는 데에 걸리는 시간 또한 상당히 절약할 수 있다. 인터넷상에서는 이미 출판되거나 게재된 뉴스나 논문, 또는 학술지 등을 통해 얻을 수도 있지만, 서로의 의견을 공유하며 답을 얻을 수 있는 커뮤니티를 통해서 얻기도 한다.

본 연구를 통해 구현하고자 하는 서비스를 구현할 때, 또 하나의 초점을 맞춘 것은 커뮤니티 기능이다. 인터넷상에 버섯 관련 커뮤니티를 검색해본 결과, 제공하고 있는 서비스의 수가 거의 없음을 확인할 수 있었다. 즉, 이러한 점이 인터넷상에서 버섯에 관련된 정보를 얻고자 할 때 많은 정보를 얻을 수 없는 원인 중 하나라고 판단하였고, 해당 문제를 개선하기 위하여 커뮤니티 기능 또한 제공하고자 설계하였다.

현재, 버섯에 관한 정보를 제공하는 커뮤니티를 검색해보면, 다양한 분야 중 버섯에 관련된 글이 올라오는 커뮤니티와 버섯에 관련된 주제만을 가지고 교류하는 커뮤니티로 나누어 살펴볼 수 있다.

다양한 분야 중 버섯에 관련된 글이 올라오는 커뮤니티 2개를 소개하고, 장단점을 분석하고자 한다. 첫 번째는 '디시인사이드의 버섯 마이너 갤러리'이다[16]. 해당 커뮤니티는 다른 커뮤니티에 비교하면 버섯에 관한 질문, 그리고 답변이 자유롭게 이루어지고 있다. 또한, 이러한 정보의 공유가 월등히 높은 것을 확인할 수 있다. 하지만, 해당 커뮤니티는 버섯이라는 분야 이외에 상당히 많은 분야를 다루고 있다는 점과 버섯의 정보 이외의 목적이 있는 사람이 주제와 맞지 않는 글을 올릴 수 있다는 문제점이 있다. 또한, 더욱 다양한 사람들이 접근하기에 제한사항이 없기에 환각성 버섯을 구하는 방법 등을 물어보는 경우도 이전에 발생한 적이 있었다는 것을 해당 커뮤니티의 공지사항을 통해 확인할 수 있다. 또 다른 하나의 커뮤니티는 '월척'이라는 커뮤니티이다[17]. '월척' 커뮤니티는 서비스의 이름에서도 알 수 있듯이, 원래는 낚시와 관련된 정보를 공유하고 나누는 서비스를 제공하고 있다. 낚시용품이나 낚시터, 월척 지식 등을 안내하고 공유하고 있지만, 자유 게시판에서는 버섯에 관련된 질문과 정보들을 조금씩 찾아볼 수 있었다. 낚시에 관련된 내용에 비하여 버섯에 관련된 내용은 상당히 적은 편이지만, 예전부터 조금씩 올라오고 있었음을 확인할 수 있다. 하지만, 해당 커뮤니티 역시 버섯에 관련된 내용만을 다루지 않는 것이 단점이라 이야기할 수 있다.

다음으로는 버섯에 관련된 주제만을 가지고 교류하는 커뮤니티이다. 첫 번째는 '머쉬타운'이라는 이름을 가지고 있는 '버섯종균닷컴'이다[18]. 여기서는 버섯을 재배할 수 있도록 정보를 제공하기도 하고, 버섯을 판매하고, 버섯을 분양하는 것이 주로 이루어지고 있다. 이 외에 자유 게시판이라는 서비스를 통하여 공유할 수 있도록 매개체가 되는 장을 제공하고 있지만, 게시된 글이 하나도 존재하지 않음을 확인하였다. 즉, 공유할 수 있는 서비스를 제공하고는 있지만 서로 간 정보의 공유가 하나도 이루어지지 않고 있기에, 어떻게 하는 것이 이러한 문제점을 극복할 수 있을지 고민하는 계기가 되었다. 또 다른 커뮤니티는 '버섯나라송송'이라는 커뮤니티이다[19]. 해당 서비스는 버섯 종균 기능사 자격을 가지고 있는 분들의 모임이라고 소개하고 있다. 해당 서비스의 취지는 '50+ 센터 내에 버섯재배 하우스를 제작하여 버섯재배를 하면서, 버섯농장 견학과 공부를 통하여 버섯재배에 대한 지식과 역량을 강화해 개인 또는 소그룹의 버섯재배가 가능할 수 있도록 함으로써 다양한 수입창출 방법을 모색하고 나아가 지역사회에 공헌하고자 한다.'고 이야기한다. 해당 커뮤니티는 전문가 또는 버섯에 관하여 자세히 탐구하고자 하는 사람들이 모여있음을 확인할 수 있으나, 커뮤니티에 참여하기 위해서는 나이 제한(만 50세~64세 미만)과 지역(서울시 거주), 그리고 인원의 제한(5명 이상)이 있기에, 개인이 참여하기에는 어려움이 많음을 확인할 수 있다. 또한, 온라인상에서는 정보의 교류는 이루어지지 않고 있다.

이 외에도 버섯에 관련된 정보를 제공하는 커뮤니티가 일부 더 있으나, 지금까지 살펴본 커뮤니티에서 발견할 수 있는 문제점과 같이 다양한 분야 중 버섯이 포함되어 있거나, 또는 정보 공유가 거의 이루어지지 않는 커뮤니티가 대부분이라는 것을 확인할 수 있다. 파악한 문제점들을 기반으로 구현하고자 하는 서비스에서는 버섯에 관련된 정보만을 얻고, 공유할 수 있도록 하고자 한다. 더 나아가서 정보의 공유가 활발하게 이루어지도록 하는 방법으로는 버섯의 전문가들에게 홍보하고 정보 제공을 요청한다면, 정보를 얻고자 하는 사용자들의 접근이 늘어나고 자연스럽게 정보 공유가 활발히 일어날 수 있을 것으로 기대한다.

3. Construction of the Dataset

3.1. Training Data Construction

본 연구에서 해결해야 하는 문제는 총 2개이다. 하나는 버섯이 어떠한 버섯인지를 확인하는 것이고, 또 다른 하나는 해당 버섯이 독버섯으로 분류되는지에 대한 여부를 확인하는 것이다. 이러한 문제를 해결하기 위한 기준이 되는 정보는 정부 기관인 농촌진흥청에서 소개하고 있는 독버섯과 야생 버섯에 대한 사진과 정보를 기준으로 하였으며, 버섯 이미지는 인터넷상에 공개된 이미지를 수집하였다[20].

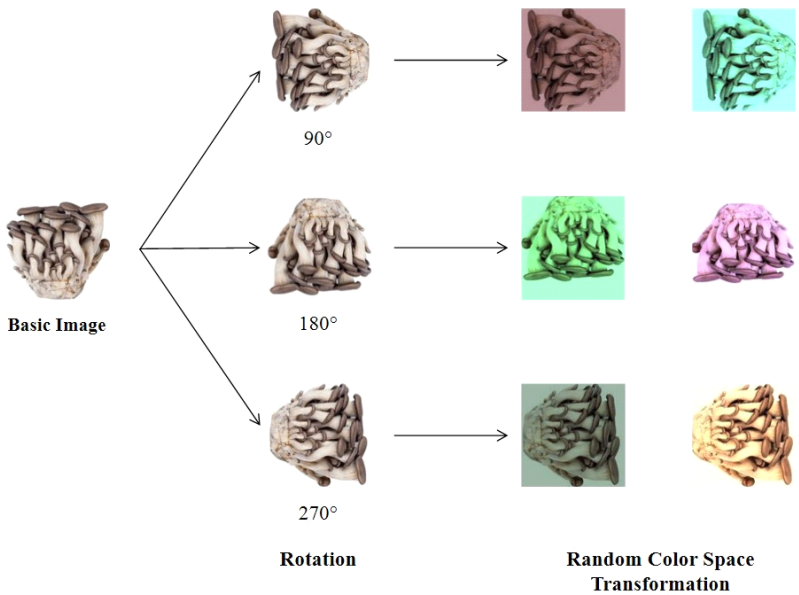
이미지를 수집할 때는 배경 정보는 고려하지 않고, 최대한 버섯의 형태가 유지되면서 다른 객체가 없는 이미지를 중점으로 수집하였다. 주어진 데이터 학습 환경이 상당히 많은 양의 이미지를 학습시킬 수 없는 제한적인 환경이었기에, 수많은 독버섯과 식용버섯 중 표본 추출하여 각 20종씩 40종을 선택하였으며, 버섯마다 50장의 원본 이미지를 수집하였다. 이러한 과정을 통해 이번 단계에서 총 2,000장의 이미지를 수집하였으며, 이를 데이터 세트로 구축하여 사용하였다.

데이터 세트를 구성하는 과정에서 본 연구가 문제를 해결하는 데 조금이나마 이바지할 수 있다고 판단하였는데, 학습에 사용하기 위한 이미지를 수집하려고 할 때 버섯에 대한 정보가 2.2.에서 설명한 바와 같이 인터넷상에 거의 없음을 다시 한번 더 확인하였다. 이러한 문제점이 인터넷에 공개된 정보를 이용하여 어떠한 버섯인가를 판단하고자 할 때 제공된 정보가 거의 없기에 잘못된 판단을 내리는 경우가 많았을 것이라 예상되었다. 본 연구에서는 해당 문제를 해결하기 위해 최대한 양질의 이미지를 수집하는 것에 초점을 맞추어 데이터 세트를 구축하였다.

3.2. Data Augmentation

본 연구에서는 위에서 소개한 선행 연구에서 직면한 한계점과 같이 데이터 세트의 크기가 작아 일반화하기에는 어려움이 있다는 것이다. 이런 문제를 해결하기 위해서 본 연구에서는 데이터 증강 기법을 적용하여 데이터 세트의 크기를 키우고자 하였다. 이를 통하여 학습되지 않는 과소 적합(Underfitting) 문제를 해결하고자 한다.

데이터 증강 기법에는 다양한 방법이 있다. Connor Shorten이 데이터 증강 기법에 관하여 정리한 것을 보면 크게 기존의 이미지를 조절하는 Basic Image Manipulations, Deep Learning을 사용하는 Deep Learning Approaches, 그리고 세부적인 기술 중 Geometric Transformations와 Neural Style Transfer를 접목한 Meta Learning이 있음을 설명하였다[21].



[Figure 1] Example: Structure of the Data Augmentation Techniques Used

각각의 갈래마다 다양한 기술들이 있지만, 본 연구에서는 기존의 이미지를 가지고 데이터 증강 기법을 사용하고자 하였기에 Basic Image Manipulations에서 Geometric Transformation을 적용하고자 하였다. Geometric Transformation은 기하학적인 변환이나 방향을 바꾸는 것, 물체의 위치를 바꾸는 것, 크기를 바꾸는 등의 여러

이미지 처리 기능이라고 설명한다. 이 중에서 이미지를 회전시키는 Rotation 기술과 이미지의 RGB 값을 조절하는 Color Space Transformation 기술을 [Figure 1]처럼 적용하였으며, 이를 통하여 더 많은 이미지를 확보하여 데이터 세트를 구축하였다. 해당 단계를 통하여 데이터를 많이 확보함으로써 더 높은 학습 정확도를 얻고자 하였다.

버섯마다 35개의 이미지를 학습에 이용하는 것으로 나누었는데, 하나의 이미지 당 9개의 이미지를 변형시켜 생성하였기에 데이터 세트의 크기가 10배 증가하였다. 즉, 데이터 증강 기법을 사용하기 전에는 1,400장의 이미지만 학습에 사용할 수 있었다면, 데이터 증강 기법을 사용한 이후 14,000장의 이미지를 학습에 사용할 수 있게 되었다. 학습 데이터 세트의 파일 크기는 61.1MB(디스크 할당: 63.8MB)에서 484MB(디스크 할당: 511MB)로 약 7.92배(디스크 할당: 약 8배) 정도 증가하였다.

데이터 세트의 크기가 증가함에 따라서 과소 적합의 문제를 조금이나마 해결할 수 있다는 장점이 있으나, 학습에 걸리는 시간이 상당히 길어진다는 단점이 있다. 본 연구는 제한된 환경에서 학습이 진행되었기에, 데이터 증강 기법을 통해 더 큰 데이터 세트를 만들어낼 수는 있었으나 하나의 이미지 당 9개의 변형된 이미지를 만들어내는 것에 그쳤다.

4. Model Experiments and Analysis

4.1. Model Architecture

구축한 데이터 세트를 학습하도록 하여 높은 판단율을 보이는 모델을 선정하고, 최종적으로 서비스에 사용하고자 한다. 이에 따라서 논문을 통해 발표된 다양한 모델 중 학습 환경이 제한됨을 고려하여, 은닉층의 개수가 너무 크거나, 연산량의 크기가 큰 모델은 제외하였다. 이 외에도 다양한 조건들에 따라 최종적으로 AlexNet[22], VGGNet[23], ResNet-50, GoogLeNet v4[24] 모델을 선정하여 초기 학습을 진행하여 최종적으로 서비스에 이용할 모델을 선정하고자 하였으며, 각 모델을 PyTorch[25]를 이용하여 구현하였다.

4.1.1. AlexNet

AlexNet은 과적합을 줄이기 위하여 두 가지 규제 기법을 적용하였다. 훈련하는 동안 Dense Layer의 출력에 Drop Out을 50%의 비율로 적용하였으며, 학습에 사용하는 이미지를 랜덤하게 여러 간격으로 이동하거나, 수평으로 뒤집고 조명을 바꾸는 식으로 데이터 증강 기술을 수행한다는 특징이 있다.

Layer	# Filters / Neurons	Filter Size	Stride	Padding	Size of Feature Map	Activation Function
Input	-	-	-	-	227 x 227 x 3	-
Convolution 1	96	11 x 11	4	-	55 x 55 x 96	ReLU
Max Pool 1	-	3 x 3	2	-	27 x 27 x 96	-
Convolution 2	256	5 x 5	1	2	27 x 27 x 256	ReLU
Max Pool 2	-	3 x 3	2	-	13 x 13 x 256	-
Convolution 3	384	3 x 3	1	1	13 x 13 x 384	ReLU
Convolution 4	384	3 x 3	1	1	13 x 13 x 384	ReLU
Convolution 5	256	3 x 3	1	1	13 x 13 x 256	ReLU
Max Pool 3	-	3 x 3	2	-	6 x 6 x 256	-
Drop Out	Rate = 0.5	-	-	-	6 x 6 x 256	-
Dense(FC) 1	4096	-	-	-	1 x 4096	-
Dense(FC) 2	4096	-	-	-	1 x 4096	-
Softmax	***40***	-	-	-	1 x ***40***	-

[Table 1] AlexNet Architecture Summary

이러한 특징을 기반으로 AlexNet 모델을 [Table 1]과 같이 구현하였다. AlexNet의 특징에서 나타난 바와 같이 Max Pool 3 Layer와 Dense(FC) 1 Layer 사이에 Drop Out을 구현하여 학습에 사용하였다. [Table 1]에 맞

추어 모델을 구축하여 요약 정보를 확인하였을 때, AlexNet은 다른 모델과는 다르게 은닉층의 수가 제일 적기 때문에 모든 지표에서 가장 작은 수치를 나타내었다. 파라미터 수는 총 62,378,344개가 나왔으며, 순전파(Propagation)와 역전파(Back-Propagation)를 수행하면서 중간에 저장되는 활성화 값과 기울기 값(Forward/Backward Pass Size)을 담는 데 필요한 메모리는 10.99MB, 학습을 위하여 필요한 메모리(Estimated Total Size)는 총 249.54MB가 필요하다고 출력되었다.

4.1.2. VGGNet

VGGNet은 많은 수의 필터를 사용하지만, AlexNet과는 다르게 3x3으로 고정된 필터만 사용하고 있다. 또한, AlexNet과 비슷하게 Convolution Layer와 Max Pool, Dense Layer로 구성되어 있으나, 은닉층의 수가 더 크다. 즉, 합성곱(Convolution) 층을 많이 사용함으로써 분류 정확도를 더욱 높이려고 한 특징이 있다.

Layer	# Filters / neurons	Filter Size	Stride	Padding	Size of Feature Map	Activation Function
Input	-	-	-	-	224 x 224 x 3	-
Convolution 1	64	3 x 3	1	1	224 x 224 x 64	ReLU
Convolution 2	64	3 x 3	1	1	224 x 224 x 64	ReLU
Max Pool 1	-	2 x 2	2	-	112 x 112 x 64	-
Convolution 3	128	3 x 3	1	1	112 x 112 x 128	ReLU
Convolution 4	128	3 x 3	1	1	112 x 112 x 128	ReLU
Max Pool 2	-	2 x 2	2	-	56 x 56 x 128	-
Convolution 5	256	3 x 3	1	1	56 x 56 x 256	ReLU
Convolution 6	256	3 x 3	1	1	56 x 56 x 256	ReLU
Convolution 7	256	3 x 3	1	1	56 x 56 x 256	ReLU
Max Pool 3	-	2 x 2	2	-	28 x 28 x 256	-
Convolution 8	512	3 x 3	1	1	28 x 28 x 512	ReLU
Convolution 9	512	3 x 3	1	1	28 x 28 x 512	ReLU
Convolution 10	512	3 x 3	1	1	28 x 28 x 512	ReLU
Max Pool 4	-	2 x 2	2	-	14 x 14 x 512	-
Convolution 11	512	3 x 3	1	1	14 x 14 x 512	ReLU
Convolution 12	512	3 x 3	1	1	14 x 14 x 512	ReLU
Convolution 13	512	3 x 3	1	1	14 x 14 x 512	ReLU
Max Pool 5	-	2 x 2	2	-	7 x 7 x 512	-
Dense(FC) 1	4096	-	-	-	1 x 4096	-
Dense(FC) 2	4096	-	-	-	1 x 4096	-
Softmax	***40***	-	-	-	1 x ***40***	-

[Table 2] VGGNet-D Architecture Summary

이러한 특징을 기반으로 VGGNet 모델을 [Table 2]와 같이 구현하였다. 이전에 설명한 바와 같이 Convolution Layer가 AlexNet에 비하여 8개가 늘어난 것을 층마다 부여한 번호를 통해 확인할 수 있다. 이 외에는 AlexNet과 비교하여 큰 차이점이 없음을 확인할 수 있다. 이를 기반으로 모델의 요약 정보를 비교하면 차이를 비교할 수 있게 되는데, VGGNet의 파라미터 수는 총 138,357,544개로 AlexNet에 비해 약 2.2배로 증가하였다. Forward/Backward Pass Size는 218.46MB로 AlexNet의 약 22배 정도 증가하였으며, Estimated Total Size는 746.83MB로 AlexNet의 약 2.9배나 증가하였다. 이를 통해 알 수 있는 사실은 Convolution Layer의 증가는 연산량 증가의 폭이 크다는 것을 확인할 수 있다.

4.1.3. GoogLeNet v4

GoogLeNet의 특징으로는 Inception Module이란 서브 네트워크를 가지고 있어, 이전의 구조(LeNet, AlexNet, VGGNet)보다 효과적으로 파라미터를 사용한다. 이 모듈은 여러 크기의 복잡한 패턴이 담긴 Feature Map을 출력하는 Convolution Layer이며, 공간상의 패턴을 잡을 수는 없지만 깊이 차원(채널)을 따라 놓인 패턴을 잡을 수 있는 특징이 있다.

GoogLeNet v4는 Inception Module과 Residual 연결을 통해 더 깊은 구조를 가지면서도 성능은 향상된 결과를 가져온 모델이다.

Layer	Filter	Filter Size	Stride	Padding	Size of Feature Map
Input					299 x 299 x 3
Stem					35 x 35 x 384
Inception-A	#Times: 4				35 x 35 x 384
Reduction-A					17 x 17 x 1024
Inception-B	# Times: 7				17 x 17 x 1024
Reduction-B					8 x 8 x 1536
Inception-C	# Times: 3				8 x 8 x 1536
Average Pooling		8 x 8			1 x 1 x 1536
Dropout	# Rate = 0.8				1 x 1536
Softmax	***40***				1 x ***40***

[Table 3] GoogLeNet v4 Architecture Summary

이러한 특징을 기반으로 GoogLeNet v4 모델을 [Table 3]과 같이 구현하였다. Inception-A, Reduction-A, Inception-B, Reduction-B, Inception-C Layer의 세부 정보는 모델의 크기가 다른 모델들과 비교하면 상당히 크기 때문에 기술하지 않았다.

모델의 요약 정보를 비교하면 차이를 비교할 수 있게 되는데, GoogLeNet v4의 파라미터 수는 총 42,711,400개로 AlexNet에 비해 약 0.32배로 감소하였다. Forward/Backward Pass Size는 422.56MB로 AlexNet의 약 38배로 증가하였으며, Estimated Total Size는 586.51MB로 AlexNet의 약 2.35배나 증가한 결과를 나타내었다.

4.1.4. ResNet-50

ResNet의 핵심 요소는 Short-cut Connection을 통한 잔차 학습(Residual Learning)이다. 그 당시까지 발표된 모델의 학습 방법은 기존의 함수 $H(x)$를 직접 근사하였다면, Redisual Function을 활용하여 $F(x) := H(x) - x$를 학습하여 학습 결과를 $F(x) + x$로 재구성한다.

Convolution Layer가 많아질수록 현재의 오차값을 줄이는 방향(Gradient)으로 파라미터를 업데이트하는데, Convolution Layer를 깊게 내려갈수록 Gradient가 너무 작아지거나 커짐으로써 학습이 더는 진행되지 못하는 Vanishing Gradient 또는 Exploding Gradient 문제를 만들어낼 수 있다. 또한, 학습 데이터를 과하게 학습하여 일반화 성능이 떨어지는 과적합(Overfitting) 문제가 발생하기도 한다.

이러한 문제를 해결하기 위하여 일반적으로 입력 x가 Layer를 통과한 출력이 목표한 값 $\hat{y}$에 맞도록 학습하나, ResNet의 경우에는 $\hat{y}$이 아닌 잔차(Residual)를 학습하도록 하였다는 특징이 있다.

Layer	# Filters / neurons	Filter Size	Stride	Padding	Size of Feature Map
Input	-	-	-	-	224 x 224 x 3
Convolution 1	64	7 x 7	2	3	112 x 112 x 64
Max Pool 1	-	3 x 3	2	1	56 x 56 x 64
Convolution 2_x	64, 64, 256	1 x 1, 3 x 3, 1, x 1	1, 1, 1	-, 1, -	56 x 56 x 256
Convolution 3_x	128, 128, 512	1 x 1, 3 x 3, 1, x 1	(2), 1, 1	-, 1, -	28 x 28 x 512
Convolution 4_x	256, 256, 1024	1 x 1, 3 x 3, 1, x 1	(2), 1, 1	-, 1, -	14 x 14 x 1024
Convolution 5_x	512, 512, 2048	1 x 1, 3 x 3, 1, x 1	(2), 1, 1	-, 1, -	7 x 7 x 2048
Average Pool 1	-	7 x 7	1	-	1 x 1 x 2048
Dense(FC) 1	1000	-	-	-	1 x 1000
Softmax	***40***	-	-	-	1 x ***40***

[Table 4] ResNet-50 Architecture Summary

이러한 특징을 기반으로 ResNet 모델을 [Table 4]와 같이 구현하였다. 모델의 요약 정보를 비교하면 차이를 비교할 수 있게 되는데, ResNet의 파라미터 수는 총 26,563,856개로 AlexNet에 비해 약 0.58배 감소하였다. Forward/Backward Pass Size는 256.13MB로 AlexNet의 약 23배 증가하였으며, Estimated Total Size는 358.04MB로 AlexNet의 약 1.43배 증가하였다. 이를 통해 잔차 학습이 파라미터 수를 효과적으로 감소시켜 효율적인 연

산이 가능하도록 하였다.

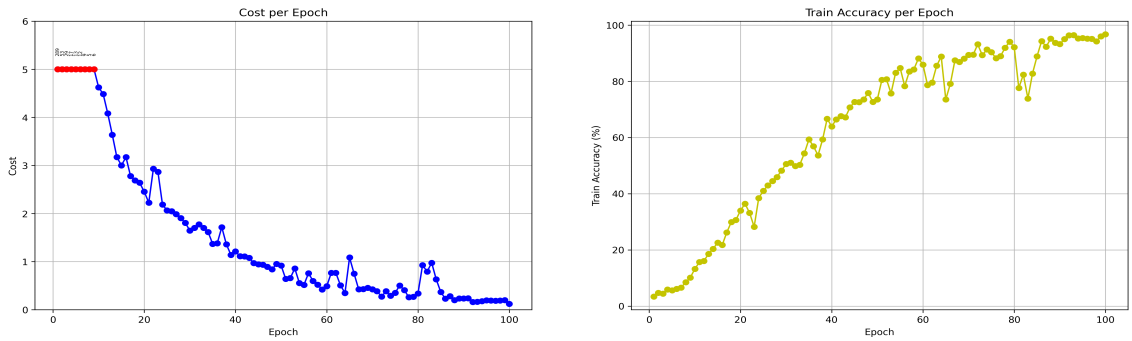
4.2. Model-Specific Training Results

OS	Windows	**Framework**	Kaggle
Python Version	3.11.11	**Optimizer**	Adam
GPU Count	1	**Learning Rate**	0.001
GPU Type	GPU P100	**Epoch**	100

[Table 5] The experimental environment and Training Model parameters

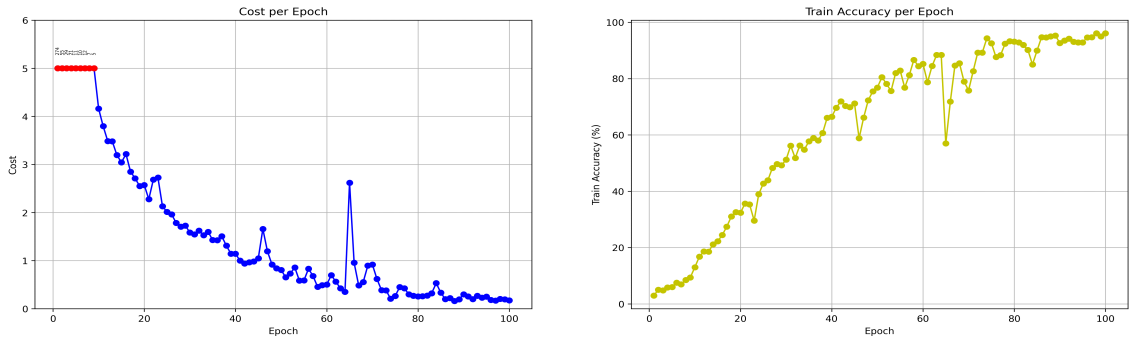
PyTorch를 이용하여 구현한 모델을 학습에 사용한 결과이다. 이는 이전에 설명한 것과 같이 구축한 이미지 세트를 학습에 이용하며, [Table 5]의 학습 환경에서 학습을 진행하였다. 학습에 사용하고자 하는 모델은 AlexNet, VGGNet, GoogLeNet, ResNet이다. 해당 모델들을 선정한 이유는 서비스를 제공할 때, 더욱 높은 정확성을 보이기 위함도 있으나, 기존 연구에서 보인 정확성보다 더욱 높은 정확도를 보이는 것을 증명하기 위하여 해당 모델을 선정하였다. [Table 1]의 환경에서 최종적으로 사용하고자 하는 모델을 선정하기 위해서 각 3번씩 학습을 진행하였으며, 학습의 결과는 다음과 같이 도출되었다.

4.2.1. AlexNet



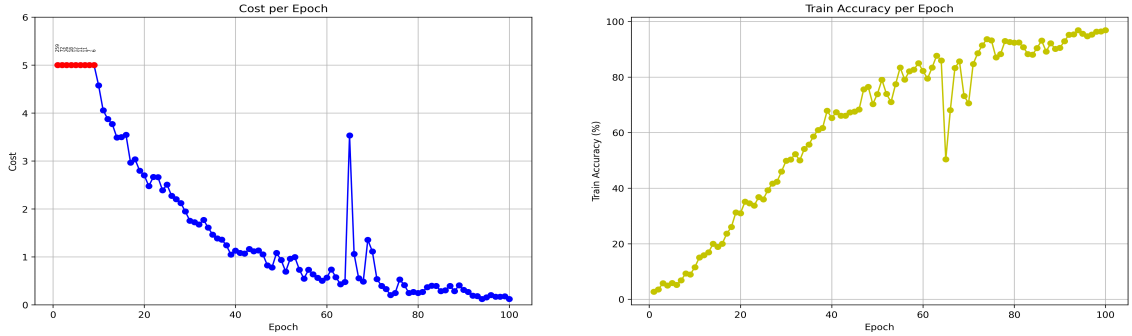
[Figure 2] AlexNet Learning Results (A) Before Data Augmentation

첫 번째 학습은 [Figure 2]를 통해 확인할 수 있는 것처럼 최종 학습 정확도는 96.77%가 나왔다. 또한, 90% 이상의 학습 정확도를 보인 시점은 72번째 학습이 완료되었을 때이다. 학습 곡선도 특이점이 없었기에 학습이 완만하게 진행되었음을 확인할 수 있다.



[Figure 3] AlexNet Learning Results (B) Before Data Augmentation

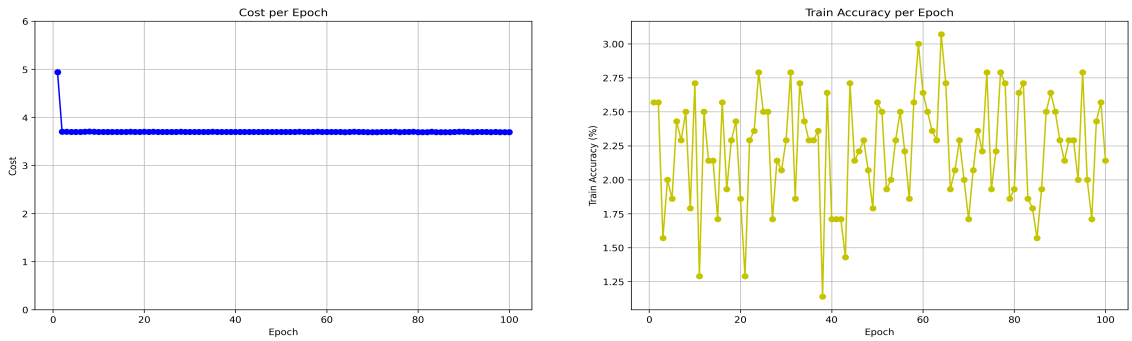
두 번째 학습은 [Figure 3]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 96.07%가 나왔다. 또한, 90% 이상의 학습 정확도를 보인 시점은 74번째 학습이 완료되었을 때이다. 학습 곡선도 특이점이 없었으나, 65번째 학습이 진행될 때 최적의 학습을 진행하기 위해 비교적 큰 조정이 진행되었음을 확인할 수 있다.



[Figure 4] AlexNet Learning Results (C) Before Data Augmentation

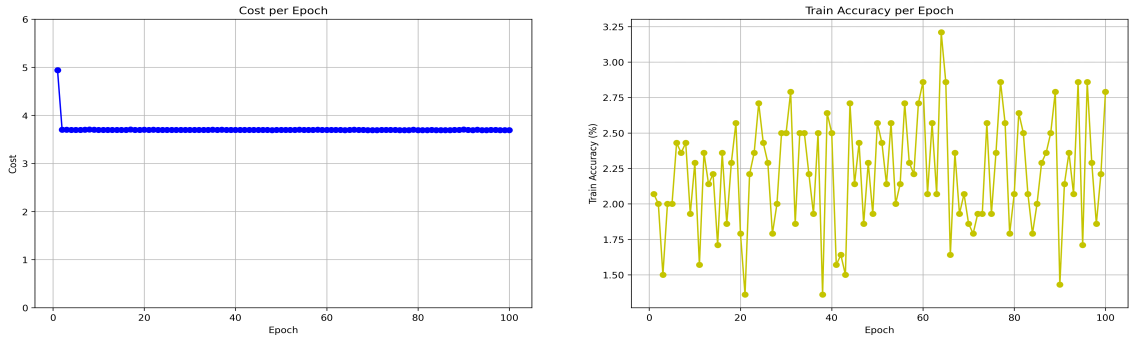
세 번째 학습은 [Figure 4]와 같이 진행되었으며, [Figure 3]와 비슷한 양상을 보였다. 최종 학습 정확도는 96.86%가 나왔으며, 90% 이상의 학습 정확도를 보인 시점은 73번째 학습이 완료되었을 때이다. 학습 곡선도 특이점은 없었으나, 65번째 학습이 진행될 때 학습을 위하여 비교적 큰 조정이 진행되었다.

4.2.2. VGGNet



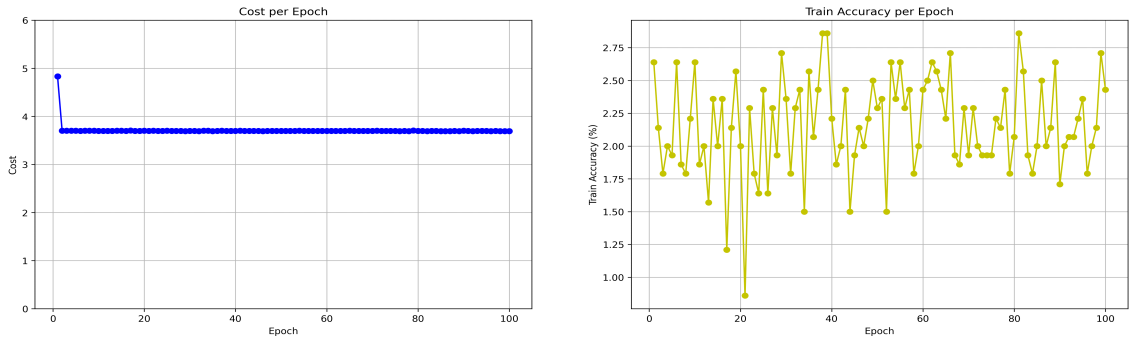
[Figure 5] VGGNet-D Learning Results (A) Before Data Augmentation

첫 번째 학습은 [Figure 5]를 통해 확인할 수 있는 것처럼 최종 학습 정확도는 3.07%가 나왔다. 학습이 진행될수록 손실 값은 낮아지고 학습 정확도가 증가하는 것이 학습이 올바르게 진행된다고 판단할 수 있으나, 이번 학습의 결과에서는 Cost의 변화가 없이 3 이상의 수치를 보여주고 있으며, 학습 정확도도 1에서 4 사이의 수치에서 비연속적인 변동 결과를 보여 학습이 진행되지 않았다고 판단하였다.



[Figure 6] VGGNet-D Learning Results (B) Before Data Augmentation

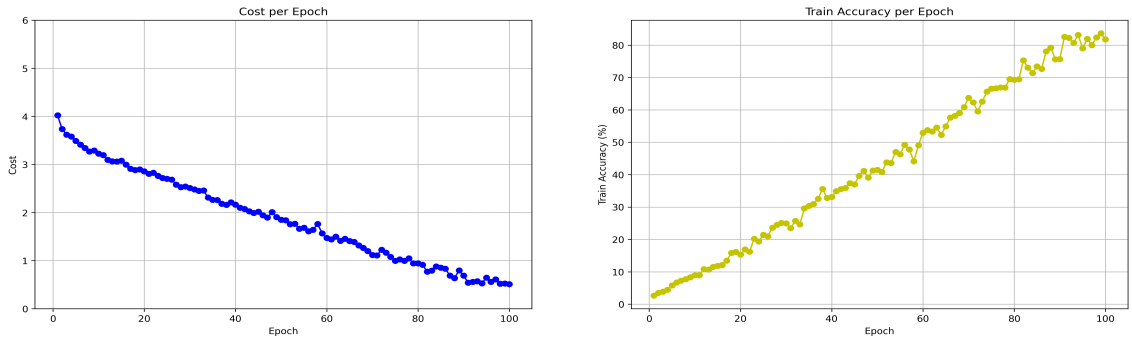
두 번째 학습은 [Figure 6]과 같이 진행되었으며, [Figure 5]와 비슷한 양상을 보였다. 최종 학습 정확도는 3.21%가 나왔으나, 이전에 설명한 바와 같이 손실 값은 내려가지 않으면서 비연속적인 학습 정확도를 보였기에, 이번 학습 역시 올바른 학습이 진행되었다고 볼 수 없다.



[Figure 7] VGGNet-D Learning Results (C) Before Data Augmentation

세 번째 학습은 [Figure 7]과 같이 진행되었으며, 이전 VGGNet 학습인 [Figure 5], [Figure 6]과 같이 손실 값의 변화는 거의 없고, 학습 정확도도 1에서 4 사이에서 비연속적으로 변화하는 모습을 보였다. 최종 학습 정확도 역시 2.86%에 그쳤기 때문에 올바른 학습이 진행되었다고 볼 수 없다.

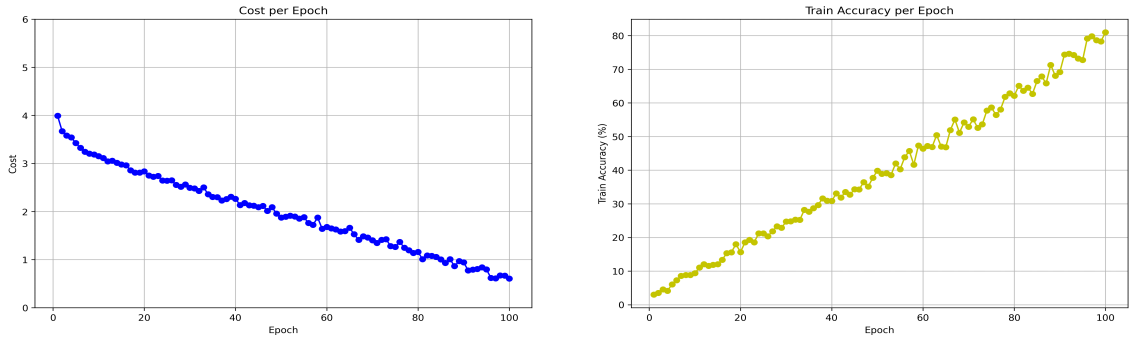
4.2.3. GoogLeNet v4



[Figure 8] GoogLeNet v4 Learning Results (A) Before Data Augmentation

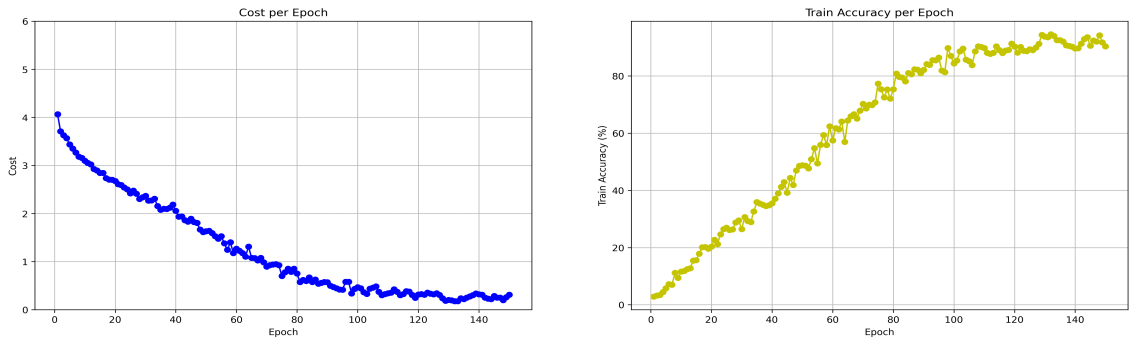
첫 번째 학습은 [Figure 8]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 83.64%가 나왔다. 그러나, 학습 진행 단계에 따른 손실 값의 추이를 살펴보면 1 이하로 떨어지는 시점이 80에서 100 사이일 때이며, 98

번째 학습이 완료되었을 때 0.52의 수치를 나타내었다. 즉, AlexNet의 학습 결과인 [Figure 2], [Figure 3], [Figure 4]와 비교하면, 더욱 정교한 학습을 위한 재조정 단계가 이루어지기 전 학습이 종료되었다. 재조정 단계가 없어도 올바른 학습이 되었다고 판단할 수 있지만, 이번 단계에서는 학습 진행 속도가 느렸기에 학습 단계가 부족하였다고 판단하였다.



[Figure 9] GoogLeNet v4 Learning Results (B) Before Data Augmentation

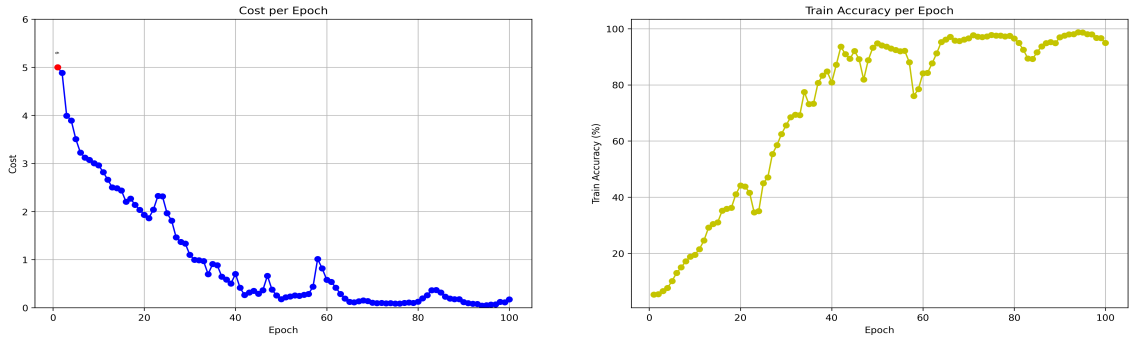
두 번째 학습은 [Figure 9]와 같이 진행되었으며, [Figure 8]과 비슷한 양상을 보였다. 최종 학습 정확도 역시 81%가 나왔으며, 손실 값이 1 이하로 떨어지는 시점이 80에서 100 사이일 때이다. 즉, 학습 속도가 [Figure 8]과 같이 느렸다고 분석할 수 있다. 그렇기에 올바른 학습이 진행되고는 있었으나, 완료되었다고는 보기 어렵기에 학습 수를 조금 늘려 학습을 한번 진행하는 방향으로 수정하였다.



[Figure 10] GoogLeNet v4 Learning Results (C) Before Data Augmentation

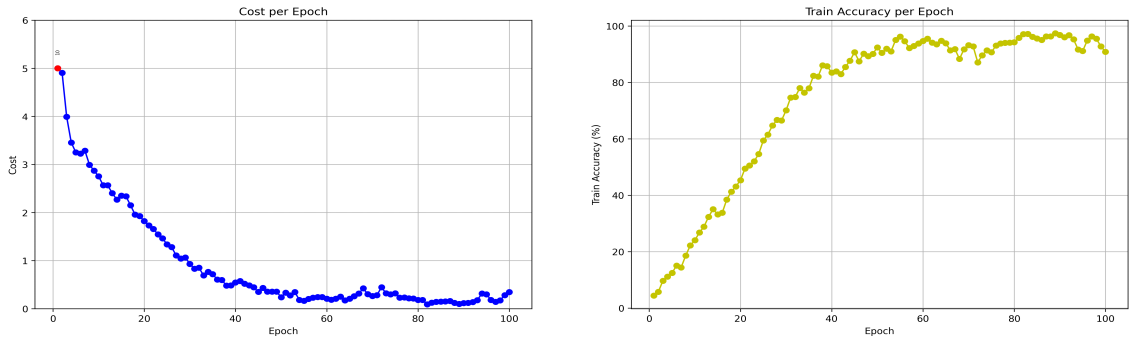
세 번째 학습은 [Figure 9]의 결과를 분석하며 설명한 바와 같이 학습 수를 100회에서 150회로 조정하여 학습을 진행하였으며 [Figure 10]의 결과를 보였다. 최종 학습 정확도는 94.57%가 나왔으며, 예상한 바와 같이 학습이 원활하게 진행되고는 있었으나 100회는 부족하여 마무리되지 않았던 것이 확인되었다.

4.2.4. ResNet-50



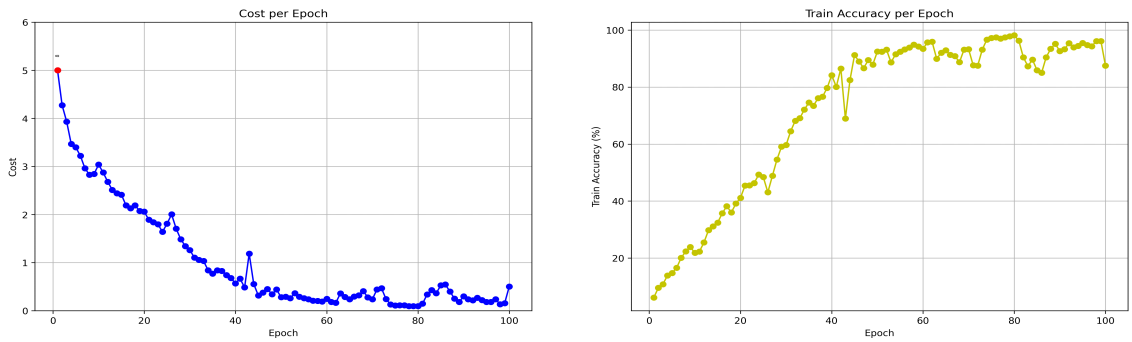
[Figure 11] ResNet-50 Learning Results (A) Before Data Augmentation

첫 번째 학습은 [Figure 11]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 94.93%가 나왔다. 또한, 90% 이상의 학습 정확도를 보인 시점은 42번째 학습이 완료되었을 때이며, 가장 높은 학습 정확도는 98.71%의 수치를 보였다. 학습 곡선도 특이점이 없었기에 학습이 완만하게 진행되었음을 확인할 수 있다.



[Figure 12] ResNet-50 Learning Results (B) Before Data Augmentation

두 번째 학습은 [Figure 12]를 통해 확인할 수 있는 것처럼 최종 학습 정확도는 90.86%가 나왔다. 또한, 90% 이상의 학습 정확도를 보인 시점은 47번째 학습이 완료되었을 때이며, 가장 높은 학습 정확도는 97.43%의 수치를 보였다. 학습 곡선도 특이점이 없었기에 학습이 완만하게 진행되었음을 확인할 수 있다. 이번 학습에서 최종 학습 정확도가 낮은 이유는 더욱 높은 정확성을 위한 조정 단계에 있을 때 학습이 마무리되어 나온 결과라고 판단하였다.



[Figure 13] ResNet-50 Learning Results (C) Before Data Augmentation

세 번째 학습의 경우에는 [Figure 13]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 87.5%가 나왔다. 이는 두 번째 학습 결과를 통해 분석한 바와 같이 조정 단계에서 학습이 마무리된 것으로 판단하였다. 이유로

는 90% 이상의 학습 정확도를 보인 시점은 45번째 학습이 완료되었을 때이며, 가장 높은 학습 정확도는 98.14%의 수치를 보였기 때문이다. 이러한 이유로 학습이 온전히 진행되었으나 최종 학습 정확도만 낮은 것으로 판단하였다.

4.3. Result Analysis

AlexNet의 경우에는 은닉층의 개수가 다른 모델과 비교하면 현저히 적기에 학습 시간이 가장 짧았다. 또한, 학습이 75번 정도 마무리되기 이전에 학습 정확도는 90%를 달성하였고, 최종 학습 정확도는 95%를 웃돌면서 좋은 결과를 보여주었다.

VGGNet의 경우에는 세 번의 학습 모두 완전한 학습이 진행되었다고 판단하기 어려운 학습 정확도를 보였다. AlexNet, ResNet, GoogLeNet의 학습이 원활하게 진행되었기에, 학습이 진행되지 않은 원인을 파악하지는 않았다. 시간적 여유가 발생하였을 때, 해당 과정을 분석하고 원인을 찾아, 재학습을 진행하여 모델 구축 단계에서 문제점이 있었는지를 확인하고자 한다. VGGNet은 수치를 기반으로 학습이 진행되지 않았다고 판단하였기 때문에 수치 분석을 진행하지 않았으며, 이번 연구에서 사용할 모델의 후보에서 제외하였다.

GoogLeNet의 경우에는 학습이 진행되고 있을 때 마무리되어 완전한 학습이 진행되었다고 판단하기에는 어려웠다. 은닉층의 관점에서 보았을 때, 다른 모델들보다는 층의 개수가 많고, 분할되었다가 합쳐지는 구조이고, 하나의 층에서도 4개의 합성곱 신경망으로 구성되었기에 학습 속도가 느릴 수밖에 없었다고 판단하였다. 또한, 연산의 양 또한 다른 모델들에 비해 상당히 많으므로 학습 시간도 상당히 길었다. 이에 따라 이번 연구에서 사용할 모델의 후보에서 제외하였다.

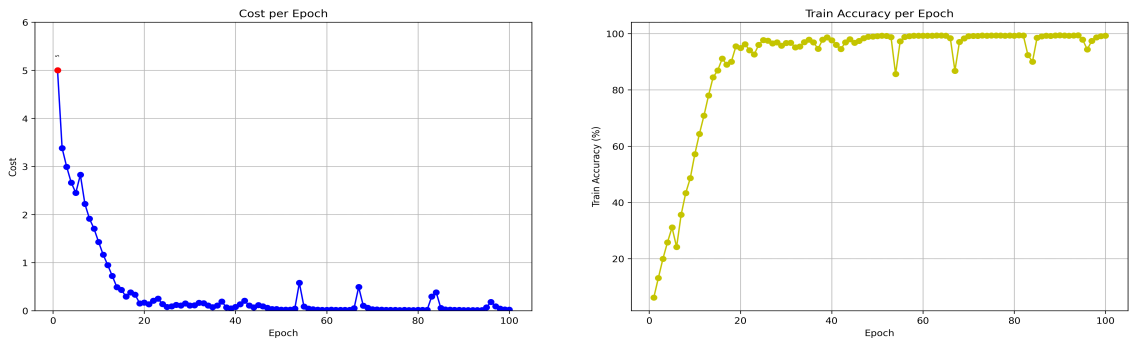
ResNet의 경우에는 학습이 50번 정도 마무리되기 이전에 학습 정확도는 90%를 달성하였고, 가장 높은 학습 정확도는 97%를 웃돌면서 좋은 결과를 보여주었다.

ResNet의 경우에는 AlexNet에 비하여 손실 값이 1 이하인 시점, 그리고 학습 정확도가 90% 이상을 처음 보인 시점이 상당히 빠르다는 것을 확인할 수 있다. 또한, 학습이 진행되는 동안 가장 높은 학습 정확도를 보인 시점의 수치를 비교해보면 AlexNet보다는 ResNet의 결과가 더욱 높다는 것을 확인할 수 있다.

이러한 수치를 기준으로 ResNet 모델에 새롭게 구축한 데이터 세트를 학습에 사용하여, 최종적으로 사용자에게 제공하고자 하는 서비스에 이용하고자 한다.

4.4. Final Training of the Model Deployed in the Service

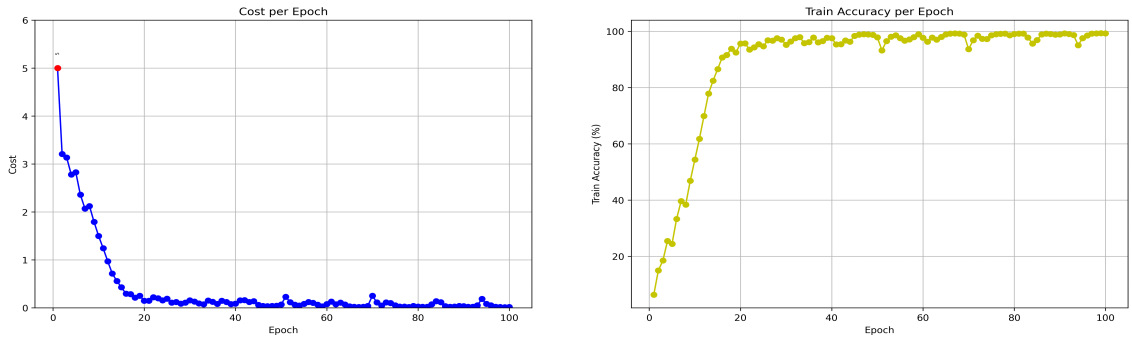
ResNet 모델을 사용하여 학습할 때, [Figure 12], [Figure 13]의 결과와 같이 조정이 이루어질 때 학습이 종료되는 것을 확인하였다. 서비스 제공에 사용될 모델을 학습할 때도 전처럼 조정이 이루어질 때 학습이 종료되는 것을 예방하기 위하여 4번의 학습을 진행하였으며, 결과는 다음과 같이 나왔다.



[Figure 14] ResNet-50 Learning Results (A) After Data Augmentation

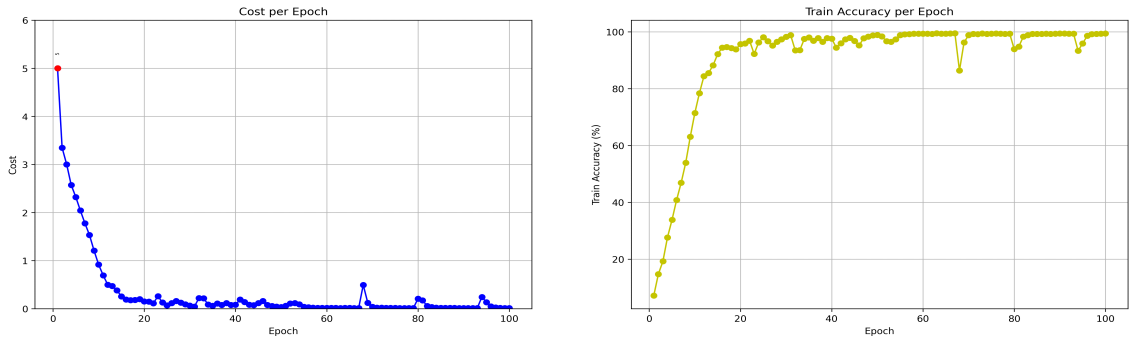
첫 번째 학습은 [Figure 14]를 통해 확인할 수 있는 것처럼 최종 학습 정확도는 99.22%가 나왔다. 또한,

90% 이상의 학습 정확도를 보인 시점은 16번째 학습이 완료되었을 때이다. 가장 높은 수치를 기록한 학습 정확도는 99.39%이다. 빠른 속도로 학습이 진행되었고, 학습 정확도를 더욱 높이기 위하여 조정도 여러 번 진행되었음을 확인할 수 있다.



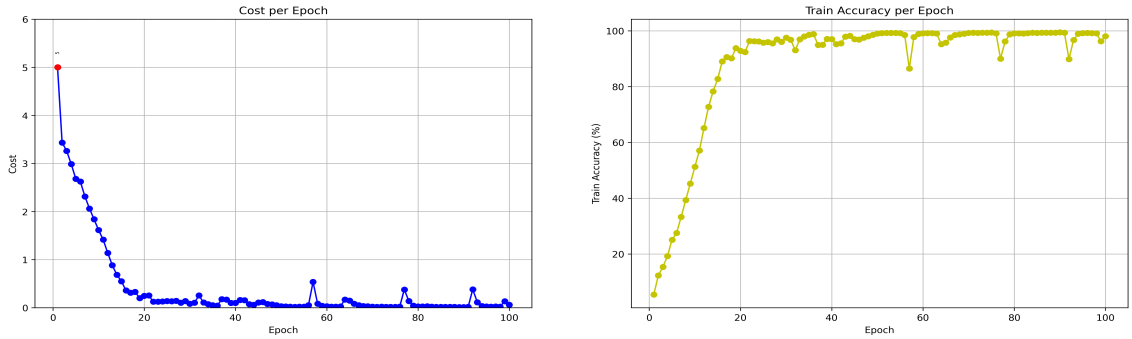
[Figure 15] ResNet-50 Learning Results (B) After Data Augmentation

두 번째 학습도 첫 번째 학습과 비슷한 양상을 보이며 [Figure 15]의 결과가 나왔다. 최종 학습 정확도는 99.31%가 나왔으며, 90% 이상의 학습 정확도를 보인 시점은 45번째 학습이 완료되었을 때이다. [Figure 14]와 비교하면 상당히 늦은 시점이라 판단할 수 있으나, 최종 학습 정확도와 손실 값을 비교한다면 학습이 잘못되었다고는 판단할 수 없다.



[Figure 16] ResNet-50 Learning Results (C) After Data Augmentation

세 번째 학습은 [Figure 16]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 99.4%로, 총 4번의 학습 중에서 가장 높은 수치가 나왔다. 또한, 90% 이상의 학습 정확도를 보인 시점은 15번째 학습이 완료되었을 때이며, 이 역시 최소한의 학습 횟수로 높은 학습 정확도를 보였다. 학습 곡선도 특이점이 없었기에 학습이 올바르게 진행되었다고 판단하였다.



[Figure 17] ResNet-50 Learning Results (D) After Data Augmentation

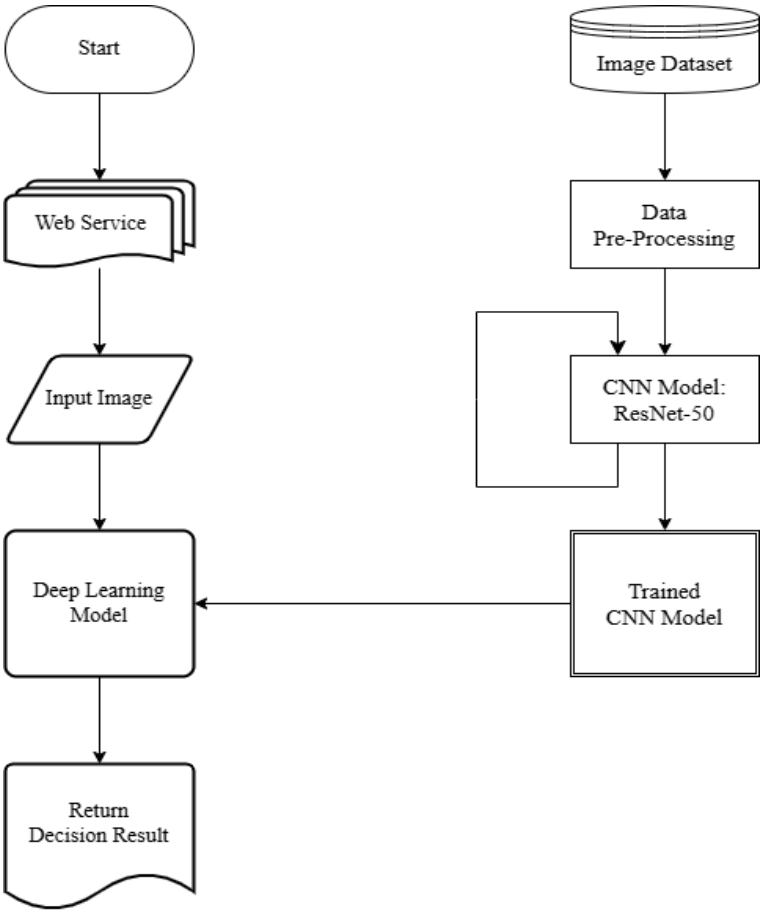
네 번째 학습은 [Figure 17]을 통해 확인할 수 있는 것처럼 최종 학습 정확도는 98.05%이며, 우려한 바와 같이 조정 이후 학습 정확도가 올라가는 중에 학습이 종료되어 다른 3개의 학습 결과와는 다르게 조금 낮은 수치를 기록하였다. 90% 이상의 학습 정확도를 보인 시점은 17번째 학습이 완료되었을 때이며, 학습 진행 과정에서 특이점은 발생하지 않았다.

4번의 학습 결과를 종합적으로 분석하면 다음과 같은 부분에서 성과를 얻었다고 판단한다. 첫 번째는 최종 학습 정확도는 98~99% 정도의 수치를 보인다는 점이다. VGGNet에서는 알 수 없는 문제로 인하여 4%의 학습 정확도를 보였지만, 98~99% 정도의 수치는 학습이 온전히 이루어졌다는 것이기에 의미 있는 성과라 판단하였다. 두 번째는 학습을 진행하는 과정 중에 가장 높은 학습 정확도는 99.3~99.5% 사이의 수치를 보였다는 것이고, 90% 이상의 학습 정확도를 보인 시점이 한 번을 제외하면 20번째 학습이 이루어지기 전에 한 번 이상이라는 것이다. 이러한 결과에는 다양한 원인이 있겠지만, 학습을 위해 설정한 학습 진행률(Learning Rate)과 최적화 알고리즘(Optimizer)이 학습에 부정적인 영향을 끼치지 않았다는 점에서도 의미 있는 결과를 얻었다고 판단하였다.

5. Design and Implementation of the Service Program

5.1. Service Architecture

지금까지 버섯을 어떻게 분류하고 판단할 것인지에 관하여 자세하게 설명하였다면, 지금부터는 학습하도록 한 모델을 사용자에게 어떠한 방식으로 서비스를 제공할 것인지에 관하여 설명하고자 한다. 제공하기 위해 구현하고자 하는 서비스는 사용자가 입력으로 제공하는 버섯 이미지를 이전에 학습시켜놓은 모델에 넣은 이후 어떠한 버섯인지를 판단토록 하여 사용자에게 다시 반환하는 비교적 간단한 구조를 갖는다.



[Figure 18] Service Architecture Flowchart

[Figure 18]은 제공하고자 하는 서비스 중, 사용자가 입력한 버섯의 이미지를 보고 어떠한 버섯인지를 판단하여 보여주는 서비스의 구조를 흐름을 순서도로 나타낸 것이다. [Figure 18]의 왼쪽은 사용자가 어떠한 흐름대로 서비스를 이용할 수 있는지를 나타내었으며, 오른쪽은 해당 서비스를 제공하기 위하여 구현한 모델의 구조를 간략하게 나타내었다. 모델의 구조 설명은 생략하며, 사용자가 어떠한 흐름대로 서비스를 이용할 수 있는지를 설명하고자 한다.

첫 번째는 'Input Image'에 해당하는 단계로, 사용자가 버섯 이미지를 올릴 수 있도록 한다. 해당 이미지는 어떠한 크기이든지 간에 제한점은 없으나, 이는 입력받은 이후 자바스크립트를 통해 모델이 요구하는 이미지 크기로 재설정하게 된다. 즉, 사용자가 올리는 버섯 이미지가 기존에 학습시켜놓은 모델의 입력 데이터가 된다.

두 번째는 'Deep Learning Model'에 해당하는 단계로, 첫 번째 단계에서 입력받은 이미지를 어떠한 버섯인지를 판단토록 하는 것이다. 사용자가 올린 버섯 이미지가 모델의 입력 데이터라고 설명하였는데, 모델의 마지막 과정에서 Softmax 함수를 사용하여 최종적으로 분류 항목에서 어떠한 항목에 근사하는지에 대한 확률을 나타내도록 하였기에 이를 사용하여 최종적으로 판단하도록 한다. 어떠한 버섯 이미지를 입력하였을 때 99% 이상의 수치를 보일 수도 있으나, 일부 사진의 경우에는 45%와 같이 신뢰하기에는 다소 부족한 수치를 보일 때도 있다. 사용자가 해당 서비스를 이용하였을 때, 최대한 많은 정보를 가지고 버섯의 정보를 판단할 수 있게 하도록 높은 순서대로 3개의 수치를 다음 단계로 전달한다.

마지막 세 번째는 'Return Decision Result'에 해당하는 단계로, 사용자에게 정보를 반환하는 단계이다. 이전 단계에서 넘어오는 수치를 예시로 간략하게 설명하면 다음과 같다. 사용자가 노루궁뎅이 버섯의 이미지를 판단토록 하였을 때, 모델은 노루궁뎅이 버섯 95%, 큰갓버섯 3%, 팽이버섯 2%로 판단하였음을 전달해준다. 이 단계에서는 전달받은 해당 정보를 사용자에게 시각적으로 보이도록 한다. 더 나아가서, 가장 높은 수치를 보인 노루궁

뎅이 버섯이 독버섯인지에 대한 여부를 판단하도록 한다. 이러한 세부적인 단계를 거쳐 마지막으로 사용자에게는 해당 사진은 노루궁뎅이 버섯으로 약 95% 정도의 판단율을 보인다고 설명하며, 해당 버섯은 독버섯이 아니라고 나타냄으로써 최종적으로 해결하고자 하는 문제에 대한 답을 사용자에게 제공한다.

이 외에 함께 제공하는 서비스는 다음과 같다. 사용자가 서비스에 접근하였을 때 단순히 어떠한 버섯인지에 대한 정보만을 얻는 것이 아닌, 해당 버섯이 어떠한 특징을 가졌는지, 독버섯이라면 어떠한 독성 성분을 가졌는지에 관한 정보를 얻을 수 있도록 버섯의 판단 정보와 함께 보일 수 있도록 제공한다. 이를 통하여 사용자가 판단 단계를 통해 얻은 버섯의 이름을 가지고, 인터넷상에 다시 검색하여 버섯의 정보를 얻는 불편함을 방지하고자 함에 있다.

더 나아가서 사용자 간의 교류할 수 있는 공간을 마련함으로써, 인터넷상에서 확보할 수 없는 버섯에 관한 다양한 정보를 올리고, 질문하고, 습득할 수 있도록 한다. 이는 본 연구와 서비스를 구현함으로써 해결하고자 했던 근본적인 원인인 정보의 한계성을 극복하는 데 이바지하고자 한다.

5.2. Methodology for the Integration of Deep Learning Models into Web Page

학습이 완료된 이후, 구현한 신경망 구조와 가중치 정보를 ONNX 확장자[26]의 형태로 저장하였다. 사용자가 어떠한 버섯인지를 알고자 할 때, 처음부터 학습이 이루어지면 오랜 시간이 소요될 수밖에 없으며, 환경의 영향도 받을 수밖에 없다. 이를 방지하기 위하여 학습을 진행하기 위해 구현한 신경망 구조와 학습을 통해 계산된 가중치 정보를 저장하였으며, 이를 이용하여 사진이 제공되면 결과가 바로 나올 수 있도록 하였다.

ONNX 확장자 파일을 불러오는 방법에는 여러 가지 방법이 있으나, 구현하기 쉬우면서도 유지와 관리가 쉽도록 서버에 함께 올려놓아 GET 방식으로 불러오는 방법을 사용하였다. 이를 통하여 사용자가 알고자 하는 버섯에 대한 사진을 넣고 버튼을 넣으면, 자바스크립트에서는 이미지 크기 조절, 모델 예측 단계를 거쳐 사용자에게 결과를 반환하는 단계로 구현하였다.

처음 웹페이지에 접근할 때 ONNX 확장자 파일이 있는지에 대한 여부를 확인한다. 이는 웹페이지 구현 코드와 같은 경로 안에만 있으면 되도록 하여, 파일을 찾지 못해 오류가 생기지 않도록 방지하도록 하였다. 이미지 크기 조절의 경우에는 모델이 지정된 초기 이미지의 크기를 요구하기에, 해당 크기에 맞추어 입력으로 들어갈 수 있도록 하였다. 마지막으로 모델 예측 단계에서는 이미 학습되어 가지고 있는 신경망 구조와 가중치 정보를 이용하여 최종적으로 어떠한 버섯을 어느 정도의 수치로 예측하는지를 나타내도록 하였다.

6. Conclusions

본 연구를 통해 해결하고자 했던 문제는 버섯에 독 성분이 포함된. 즉, 독버섯인지에 관한 여부를 판단하고자 함에 있다. 제시한 문제를 해결하고자 한 원인은 현재 독버섯인지에 관한 판단을 위해서 다양한 정보를 기반으로 추론하고 판단해야 하지만, 다양한 정보에 해당하는 정보의 양이 지금까진 제한적이었다고 판단하였기 때문이다. 가장 접근하기 쉬운 인터넷상에서 정보를 많이 얻을 수 있을 것으로 생각하였으나, 인터넷상에는 다른 분야에 비해 상당히 적은 양의 정보만을 보여주고 있다는 것이다. 해당 원인은 기존의 선행 연구에서도 같은 문제로 거론되었으며, 본 연구를 진행하는 과정에서 데이터 세트를 수집하는 과정에서도 같은 문제를 겪었다.

버섯의 이미지는 인터넷상에 공개된 자료를 수집하였으나, 자료의 양이 적기에 인공지능을 학습시키기엔 부족하였다. 이를 극복하기 위하여 데이터 증감 기법을 적용하여 데이터 세트의 양을 상당히 많이 늘렸으나, 그런데도 자료수집에 대한 한계점을 완전히 극복하였다고는 볼 수 없다.

모델 학습의 관점에서는 ResNet-50을 구현하여 학습에 사용하였고, 이를 4번으로 나누어 각각 학습을 진행

하여 일반화된 결과를 얻고자 하였다. 학습 진행 중에서는 학습 정확도의 차이는 보이지만, 학습이 마무리되어 가는 과정을 분석하면 95% 이상인 약 99% 정도의 학습 정확도를 보이면서 학습이 원활하게 잘 마무리되었음을 확인할 수 있다.

그러나, 본 연구의 또 다른 문제로 학습에 사용할 수 있는 환경에 제한이 있었다는 것을 고려하여, 학습에 사용된 이미지의 수는 14,000장의 이미지와 40개의 항목으로만 나누어져 있기에 이를 실생활에 바로 적용하기엔 상당히 많은 오류와 문제점이 발생할 것으로 생각된다. 그렇다고 해서 해당 연구가 의미가 없다고 판단할 수는 없다. 이는 앞으로 학습 환경에 제한사항이 없다면 데이터 증강 기법을 적용하여 더 큰 데이터 세트를 구축할 수 있고, 더 많은 항목으로 구축하면 실생활에 적용할 수 있다는 것이다. 이를 이용하면 실생활에서 바로 사용할 수 있는 유의미한 결과로 도달될 것으로 기대한다.

서비스의 관점에서는 인터넷상에서 정보를 얻는데 한계점이 명확하다는 것을 해결하고자 하였다. 가장 중점이 되는 서비스는 사용자가 버섯의 이미지를 입력하였을 때, 어떠한 버섯인지와 독버섯인지에 관한 정보를 제공하는 것이지만, 이에 그치지 않고 커뮤니티 서비스를 제공하여 다양한 사람 간 교류가 이루어질 수 있도록 하나의 매개체 역할을 할 수 있는 서비스로 구현하였다. 이를 통하여 버섯에 관하여 탐구하는 교수 및 전문가와 버섯을 전문적으로 채취하는 전문가, 그리고 호기심을 가지고 있는 학생 등 다양한 분야의 사용자가 커뮤니티 서비스를 통하여 교류할 수 있기에 지금보다 더욱 많은 정보가 생산되고 공유될 수 있다고 생각한다. 이를 통해 본질적인 문제인 인터넷상에서 정보를 수집하는 것에 관한 어려움을 해결할 수 있을 것으로 기대한다. 또한, 제공하는 서비스를 통하여 독버섯에 의한 사건과 사고 발생 수가 줄어들 수 있을 것으로 생각한다.

7. References

[1] “[출근길 인터뷰] 가을 산행 '독버섯' 주의보…"모르면 먹지 마세요",” 연합뉴스TV, Oct. 21, 2024. [Online]. Available: https://www.yonhapnewstv.co.kr/news/MYH20241021003200641?input=1825m. [Accessed: Mar. 13, 2025].

[2] “[집중인터뷰] 독버섯·실족·낙상…가을 산행 사고 주의보,” KBS 뉴스, Oct. 31, 2024. [Online]. Available: https://news.kbs.co.kr/news/pc/view/view.do?ncd=8095500&ref=A. [Accessed: Mar. 13, 2025].

[3] 행정안전부, "무더운 여름철, 야생버섯 섭취 주의!," 농촌진흥청, Jun. 24, 2024. [Online]. Available: https://www.rda.go.kr/board/board.do?boardId=farmprmninfo&prgId=day_farmprmninfoEntry&mode=updateCnt&searchOrgDeptKey=org&dataNo=100000796595. [Accessed: Mar. 13, 2025].

[4] 산림청, “우리 나라 버섯 2,170종 중 먹을 수 있는 것은 493종 뿐...가을 독버섯 주의,” 산림청, Sep. 25, 2023. [Online]. Available: https://www.forest.go.kr/kfsweb/cop/bbs/selectBoardArticle.do?bbsId=BBSMSTR_1036&mn=NKFS_04_02_01&nttId=3187724. [Accessed: Mar. 13, 2025].

[5] M. Kim, S. Jeon, J. Lee, S. Hong, J. Bang, and H. Kim, “Study of Classifying mushroom images using CNN-based models,” in *Proc. Korean Institute of Communications and Information Sciences (KICS) Conf.*, Gangwon, South Korea, Feb. 2022, pp. 1289–1290.

[6] “Mushrooms classification – Common genus’s images,” Kaggle, n.d. [Online]. Available: https://www.kaggle.com/datasets/maysee/mushrooms-classification-common-genuss-images. [Accessed: Apr. 29, 2025].

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770-778.

[9] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

[10] D. Yoon, “A Study on the Implementation of a Poisonous Mushroom Determination Program through Deep Learning,” in Proc. of the Korea Institute of Information Scientists and Engineers (KIISE) Academic Conference, Jeju, South Korea, Dec. 20, 2022.
[11] A. F. Agarap, “Deep learning using rectified linear units (ReLU),” arXiv preprint arXiv:1803.08375, 2018.
[12] D. A. Clevert, T. Unterthiner, and S. Hochreiter, “Fast and accurate deep network learning by exponential linear units (ELUs),” arXiv preprint arXiv:1511.07289, 2015.
[13] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv preprint arXiv:1412.6980, 2014.
[14] G. Hinton, N. Srivastava, and K. Swersky, “The Momentum Method,” in Neural Networks for Machine Learning, Lecture 6c, Dept. of Computer Science, Univ. of Toronto. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf [Accessed Apr. 29, 2025].
[15] Glority, “Picture Mushroom,” Google Play Store (Mobile App), n.d. [Online]. Available: https://play.google.com/store/apps/details?id=com.glority.picturemushroom&hl=ko. [Accessed: May 21, 2025].
[16] DCInside, “Mushrooms Gallery,” DCInside. [Online]. Available: https://gall.dcinside.com/mgallery/board/lists/?id=mushrooms. [Accessed: May 21, 2025].
[17] Wolchuck, “Wolchuck Community,” Wolchuck Community, n.d. [Online]. Available: https://www.wolchuck.co.kr/. [Accessed: May 21, 2025].
[18] MushTown Agricultural Cooperative, “Mushroom Spawn.com,” MushTown Agricultural Cooperative, n.d. [Online]. Available: https://mushtown.net/. [Accessed: May 21, 2025].
[19] 50plus, “Community Home,” 50plus, n.d. [Online]. Available: https://www1.50plus.or.kr/sdc/community-home.do?id=33161454. [Accessed: May 21, 2025].
[20] “버섯정보포털,” 농촌진흥청, n.d. [Online]. Available: https://www.nihhs.go.kr/mushroom/psnMush/content0101.do. [Accessed: Mar. 15, 2025].
[21] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” Journal of Big Data, vol. 6, no. 1, pp. 1–48, 2019.
[22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” Commun. ACM, vol. 60, no. 6, pp. 84–90, 2017.
[23] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv preprint arXiv:1409.1556, 2014.
[24] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, “Inception-v4, Inception-ResNet and the impact of residual connections on learning,” in Proc. AAAI Conf. Artif. Intell., vol. 31, no. 1, Feb. 2017.
[25] “PyTorch,” PyTorch, n.d. [Online]. Available: https://pytorch.org/. [Accessed: Apr. 22, 2025].
[26] “torch.onnx — PyTorch 2.6 documentation,” PyTorch Documentation, 2024. [Online]. Available: https://pytorch.org/docs/stable/onnx.html. [Accessed: Apr. 20, 2025].

Appendix A: Mid-Term Q&A

(All questions were handled Anonymously.)

Q. 갤럭시의 AI 이미지 읽는 기술을 사용해서 버섯을 찍어서 "이게 무슨 버섯이야?"라고 물었을 때 "어떤 어떤 버섯입니다“ 하면서 인터넷 페이지를 보여주면서 잘 알려주는데 -> 이런 AI와 차별점? 다른 점이 있을까요?

A. 갤럭시가 제공하는 AI의 학습 데이터, 학습 환경을 넘어서서 제공할 수 있는 서비스는 없습니다.
하지만, 이와는 다르게 전문가와 사용자 간의 매개체가 될 수 있도록 하나의 큰 서비스로 만들어낸다면, 갤럭시의 AI와는 차별성이 있을 것으로 예상합니다.

Q. 각 50장씩 모으셨다고 했는데 정확도는 어떻게 되나요?

A. 본 연구를 통해 해결하고자 하는 이유가 해당 질문의 취지와 연결 지어서 설명할 수 있습니다.
데이터 세트를 구축하기 위해 인터넷상에 공개된 버섯의 이미지를 수집하려 하였으나, 사실상 50장을 모으기도 어려운 것이 현실입니다. 즉, 인터넷을 통해 버섯의 종류를 판단하기에는 어렵기에 해당 문제를 해결하고자 하는 것입니다.
이를 위하여 데이터 증강 기법을 적용하여 데이터 세트의 크기를 키웠습니다. 원본 이미지 대비 10배 정도의 데이터 세트를 구축하였으며, 현재 학습 정확도의 경우에는 95% 이상 되도록 하고 있습니다.

Q. 붉은사슴뿔버섯같은 경우에는 전문가도 육안으로 구분이 힘들 정도로 동충하초나 영지버섯하고 유사한데 이미지로만 구분하면 위험하지 않나요?

A. 해당 질문은 조금 쉽게 생각한다면, 붉은사슴뿔버섯에 대한 가중치를 높여 학습에 적용토록 하면 해결될 문제라고 생각합니다.

Q. 식물의 이미지를 보여주면 어떤 식물인지 나오는 앱이 이미 있는 것으로 아는데 버섯으로 한정한 이유가 있을까요?

A. 두 번째 질문과 같은 답변으로 대체하면 될 것 같습니다. 단, 식물 전체보다는 버섯 데이터에 특화된 서비스라는 것에서 차별점이 있을 듯합니다.

Q. 오류로 인해서 독버섯을 식용버섯으로 인식해 버린 경우 책임은 누가 지나요?

A. 해당 문제에 대해서는 책임 소재를 서비스를 제공하는 저희가 온전히 부담하기엔 어렵습니다.
유사한 서비스의 경우, 심지어는 Chat GPT도 오류를 발생한다고 지속해서 알리고 있기에 해당 부분에 대해서는 생각하지 않았으나, 높은 정확도를 위하여 지속적인 유지 보수를 진행해나갈 예정입니다.

Q. 식용버섯이지만 잘못된 조리법으로 독버섯이 되는 경우는 어떡하나요(예를 들어 팽이버섯)

A. 해당 서비스는 단순히 어떠한 버섯인지를 판단하는 서비스입니다.
따라서, 해당 버섯을 조리, 가공 단계를 거친 이후의 판단에 대해서는 고려하지 않습니다.

Q. 보통 산에서 이용할 것 같은데, 인터넷이 안되면 서비스 자체가 안될 텐데 앱으로 구현 하지 않는 이유가 있나요?

A. 앱으로 구현할 예정입니다.

Q. 실제로 버섯을 야생에서 채집하는 인구가 한국에는 정말 적고 대부분 고령층으로 알고 있는데 그 연령층은 AI 같은 부분에 크게 관심이 없는 것 같은데 이 부분에 대한 생각이 궁금합니다

A. 간단한 접근 방식을 적용하고자 합니다.
단순하게 사진을 찍고, 버튼을 누르면 해당 버섯에 대한 정보가 나오기 때문에 진입 장벽은 낮을 것으로 생각합니다.

Q. 대부분 버섯 섭취는 마트나 이미 식용 가능한 버섯들만 유통하는데 실용성이 있나요?

A. 해당 문제를 해결하고자 하는 취지는 임산물들을 채취하는 과정에서 발생하는 사건 사고를 예방함에 있습니다. 즉, 마트나 플랫폼을 이용하여 버섯을 구매하는 과정에 대해서는 고려할 대상이 아닙니다.

Appendix B: Final-Term Q&A

(All questions were handled Anonymously.)

Q. 이 서비스를 사용할 때 사람들이 자주 사용할 것 같나요?
보통 사람들은 산에서 수상한 버섯을 먹으려 하지 않아서 극히 일부만 사용할 것 같습니다.
A. 인공지능을 통한 버섯 판단만이 목적이 아닌 버섯 커뮤니티를 만들어갈 수 있다면,
해당 기능으로도 좋은 서비스를 구현했다고 판단할 수 있을 것 같습니다.

Q. 고사리도 독이 있는데 버섯도 독이 있어도 식용으로 먹을 수 있다면 독버섯으로 판단하나요? 식용버섯으로 판단하나요.
A. 대상은 버섯으로 분류되는 것만을 기준으로 하기에, 고사리는 고려 대상이 아닙니다.

Q. 데이터 증강이 독버섯 구분에 오히려 혼란을 줄 수 있는 거 아닌가요.
버섯의 독성 여부는 색감 무늬 갓의 윤곽 등 미세한 특징으로 구분되는 경우가 많고 일반적인 식용버섯이나 독버섯도
자라는 환경이나 그 지역의 강수량 햇빛 배지 등 다양한 환경요건에 영향을 받아 생김새가 달라지는 경우가 많은데
사용자의 사진으로 밝기나 채도 조정 같은 증강이 핵심 판단 요소를 왜곡할 가능성이 없나요?
A. CNN의 분류는 단순히 인간이 이해하고 분류하는 것과는 다른 방식으로 이해하고 분류합니다.
버섯의 형태가 변화한 경우는 다른 모델에서도 똑같이 판단에 오류를 보이는 경향이 있을 것으로 생각됩니다.
사용한 증강 기술은 이미 많이 사용되고 있으며, 이로 인하여 문제가 발생한다고 판단할 수는 없을 것입니다.

Q. 온전한 버섯이 아닌 파손되거나 일부가 잘려 형태를 알아보기 힘든 상태의 버섯은 어떻게 구분하나요.
A. 모든 모델이 그렇듯이 온전한 버섯이 아닌 경우에는 판단에 오류를 나타낼 가능성이 큽니다.

Q. 만약 독버섯과 일반 버섯이 50대 50의 결과로 나온다면 어떻게 출력되나요?
A. 50대 50의 결과가 나오기는 어려우나, 모델에서 Top-1으로 도출된 Class의 기준에 맞추어 출력됩니다.

Q. 데이터 세트가 적으면 과적합 위험이 크지 않나요? 모델을 구현할 때 관련 설정을 어떻게 하셨나요?
A. 과적합을 방지하기 위해 데이터 증강기법을 적용하였으며, 모델 설정은 보고서에 자세하게 기술하였습니다.

Q. 독버섯은 전문가도 구분하기 힘든데, 학습되는 데이터들은 개발자가 독버섯인가 아닌가로 구분하고 판단해서
학습시켜야 하는데 만일 잘못된 데이터를 학습시켜서 AI 오류로 사망 사고가 일어나게 된다면
책임은 누구에게 물어야 하나요?
A. 윤리적 문제는 AI가 적용된 기술과 서비스에 모두 적용되는 문제입니다.
구현한 본 서비스도 판단에 도움을 주는 보조 수단일 뿐, 절대적인 맹신을 하지 않도록 안내하고 있습니다.

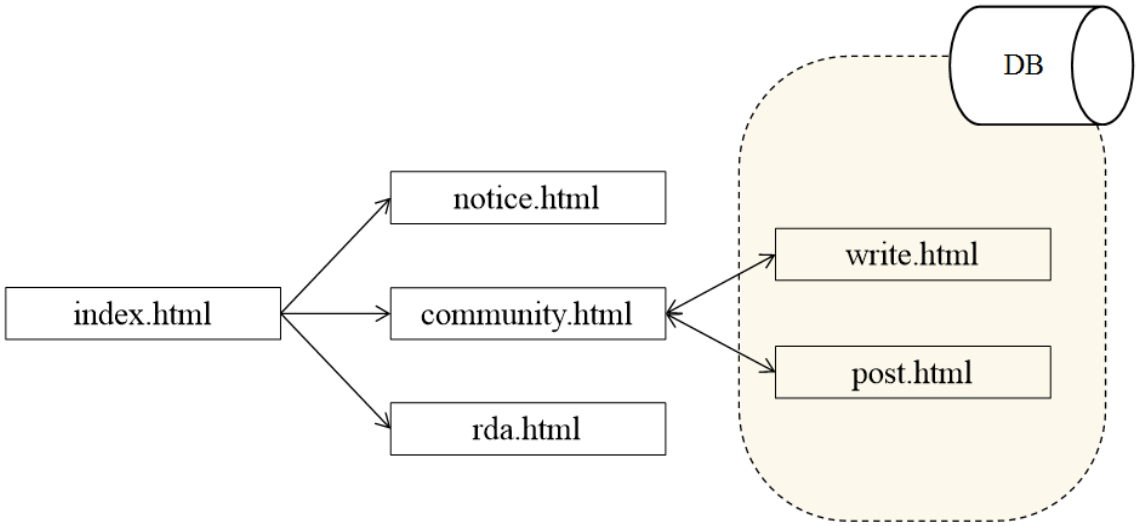
Q. 전문가 감수 없이 AI의 판단만으로 결과를 제시하는 것이 위험하다고 느낀 적은 없으셨나요?
그리고 생태계 보호 측면에서 채취를 유도하는 방향성이 문제 될 소지는 없을까요?
A. 위와 같은 답변이 해답이 될 것 같습니다. 덧붙이자면, 버섯에 관한 정보를 얻고자 한다면 사진을 찍어 확인할 수 있도록
하고 있기에 채취를 유도하고 있지는 않습니다.

Appendix C: Changes in Cost and Training Accuracy by Epoch

[Epoch: 1] cost = 5.03785753 | Train Accuracy = 7.18%
[Epoch: 2] cost = 3.34811044 | Train Accuracy = 14.73%
[Epoch: 3] cost = 3.00165701 | Train Accuracy = 19.25%
[Epoch: 4] cost = 2.57097173 | Train Accuracy = 27.65%
[Epoch: 5] cost = 2.32290983 | Train Accuracy = 33.86%
[Epoch: 6] cost = 2.04203963 | Train Accuracy = 40.82%
[Epoch: 7] cost = 1.77672935 | Train Accuracy = 46.88%
[Epoch: 8] cost = 1.53286076 | Train Accuracy = 53.94%
[Epoch: 9] cost = 1.20875525 | Train Accuracy = 63.07%
[Epoch: 10] cost = 0.918410599 | Train Accuracy = 71.45%
[Epoch: 11] cost = 0.695296705 | Train Accuracy = 78.42%
[Epoch: 12] cost = 0.499519378 | Train Accuracy = 84.43%
[Epoch: 13] cost = 0.473792166 | Train Accuracy = 85.49%
[Epoch: 14] cost = 0.382705867 | Train Accuracy = 88.24%
[Epoch: 15] cost = 0.257769763 | Train Accuracy = 92.14%
[Epoch: 16] cost = 0.190027907 | Train Accuracy = 94.41%
[Epoch: 17] cost = 0.17741701 | Train Accuracy = 94.66%
[Epoch: 18] cost = 0.183287129 | Train Accuracy = 94.30%
[Epoch: 19] cost = 0.19839859 | Train Accuracy = 93.88%
[Epoch: 20] cost = 0.150666609 | Train Accuracy = 95.65%
[Epoch: 21] cost = 0.148949102 | Train Accuracy = 95.93%
[Epoch: 22] cost = 0.111455247 | Train Accuracy = 96.86%
[Epoch: 23] cost = 0.262423187 | Train Accuracy = 92.22%
[Epoch: 24] cost = 0.129081413 | Train Accuracy = 96.29%
[Epoch: 25] cost = 0.0720255449 | Train Accuracy = 98.07%
[Epoch: 26] cost = 0.116422668 | Train Accuracy = 96.69%
[Epoch: 27] cost = 0.159173116 | Train Accuracy = 95.19%
[Epoch: 28] cost = 0.127393126 | Train Accuracy = 96.49%
[Epoch: 29] cost = 0.0895423293 | Train Accuracy = 97.39%
[Epoch: 30] cost = 0.0638006479 | Train Accuracy = 98.23%
[Epoch: 31] cost = 0.0456226207 | Train Accuracy = 98.83%
[Epoch: 32] cost = 0.223170817 | Train Accuracy = 93.51%
[Epoch: 33] cost = 0.217727557 | Train Accuracy = 93.54%
[Epoch: 34] cost = 0.0891453475 | Train Accuracy = 97.51%
[Epoch: 35] cost = 0.0615138002 | Train Accuracy = 98.06%
[Epoch: 36] cost = 0.109052725 | Train Accuracy = 96.88%
[Epoch: 37] cost = 0.0768860653 | Train Accuracy = 97.79%
[Epoch: 38] cost = 0.116180651 | Train Accuracy = 96.47%
[Epoch: 39] cost = 0.0785805658 | Train Accuracy = 97.84%
[Epoch: 40] cost = 0.0821601674 | Train Accuracy = 97.61%
[Epoch: 41] cost = 0.191546261 | Train Accuracy = 94.45%
[Epoch: 42] cost = 0.13973546 | Train Accuracy = 95.99%
[Epoch: 43] cost = 0.0841861069 | Train Accuracy = 97.39%
[Epoch: 44] cost = 0.0742468908 | Train Accuracy = 97.87%
[Epoch: 45] cost = 0.118665941 | Train Accuracy = 96.78%
[Epoch: 46] cost = 0.163004041 | Train Accuracy = 95.21%
[Epoch: 47] cost = 0.0767698288 | Train Accuracy = 97.74%
[Epoch: 48] cost = 0.0572732165 | Train Accuracy = 98.35%
[Epoch: 49] cost = 0.0451983102 | Train Accuracy = 98.74%
[Epoch: 50] cost = 0.0350177288 | Train Accuracy = 98.92%
[Epoch: 51] cost = 0.0592477359 | Train Accuracy = 98.40%
[Epoch: 52] cost = 0.108808018 | Train Accuracy = 96.70%
[Epoch: 53] cost = 0.117434785 | Train Accuracy = 96.52%
[Epoch: 54] cost = 0.0922686085 | Train Accuracy = 97.36%
[Epoch: 55] cost = 0.0393362269 | Train Accuracy = 98.84%
[Epoch: 56] cost = 0.0319657065 | Train Accuracy = 99.12%
[Epoch: 57] cost = 0.0234413184 | Train Accuracy = 99.20%
[Epoch: 58] cost = 0.0179055259 | Train Accuracy = 99.33%
[Epoch: 59] cost = 0.016184371 | Train Accuracy = 99.34%
[Epoch: 60] cost = 0.0168305524 | Train Accuracy = 99.34%
[Epoch: 61] cost = 0.0176926162 | Train Accuracy = 99.32%
[Epoch: 62] cost = 0.0166131426 | Train Accuracy = 99.26%
[Epoch: 63] cost = 0.0143836522 | Train Accuracy = 99.48%
[Epoch: 64] cost = 0.0165970773 | Train Accuracy = 99.38%
[Epoch: 65] cost = 0.0159529466 | Train Accuracy = 99.35%
[Epoch: 66] cost = 0.0133582111 | Train Accuracy = 99.41%
[Epoch: 67] cost = 0.014260455 | Train Accuracy = 99.47%
[Epoch: 68] cost = 0.494880557 | Train Accuracy = 86.37%
[Epoch: 69] cost = 0.120781347 | Train Accuracy = 96.27%
[Epoch: 70] cost = 0.0391044654 | Train Accuracy = 98.82%
[Epoch: 71] cost = 0.0224948507 | Train Accuracy = 99.26%
[Epoch: 72] cost = 0.02286865 | Train Accuracy = 99.21%
[Epoch: 73] cost = 0.0165454354 | Train Accuracy = 99.40%
[Epoch: 74] cost = 0.0177481398 | Train Accuracy = 99.25%
[Epoch: 75] cost = 0.0166376159 | Train Accuracy = 99.35%
[Epoch: 76] cost = 0.0140445391 | Train Accuracy = 99.40%
[Epoch: 77] cost = 0.0154282553 | Train Accuracy = 99.33%
[Epoch: 78] cost = 0.0151336892 | Train Accuracy = 99.29%
[Epoch: 79] cost = 0.0158322323 | Train Accuracy = 99.36%
[Epoch: 80] cost = 0.209734619 | Train Accuracy = 93.92%
[Epoch: 81] cost = 0.175947517 | Train Accuracy = 94.78%
[Epoch: 82] cost = 0.0577297211 | Train Accuracy = 98.32%
[Epoch: 83] cost = 0.0358809158 | Train Accuracy = 98.85%
[Epoch: 84] cost = 0.0207681339 | Train Accuracy = 99.27%
[Epoch: 85] cost = 0.017615471 | Train Accuracy = 99.27%
[Epoch: 86] cost = 0.0164034739 | Train Accuracy = 99.27%
[Epoch: 87] cost = 0.0175686963 | Train Accuracy = 99.35%
[Epoch: 88] cost = 0.0180980116 | Train Accuracy = 99.24%
[Epoch: 89] cost = 0.0144086136 | Train Accuracy = 99.38%
[Epoch: 90] cost = 0.0137632815 | Train Accuracy = 99.40%
[Epoch: 91] cost = 0.0125420177 | Train Accuracy = 99.43%
[Epoch: 92] cost = 0.0134631842 | Train Accuracy = 99.35%
[Epoch: 93] cost = 0.0144357989 | Train Accuracy = 99.35%
[Epoch: 94] cost = 0.244474858 | Train Accuracy = 93.32%
[Epoch: 95] cost = 0.136757344 | Train Accuracy = 95.93%
[Epoch: 96] cost = 0.0466746762 | Train Accuracy = 98.58%
[Epoch: 97] cost = 0.0261333864 | Train Accuracy = 99.20%
[Epoch: 98] cost = 0.0184880309 | Train Accuracy = 99.29%
[Epoch: 99] cost = 0.0141664073 | Train Accuracy = 99.38%
[Epoch: 100] cost = 0.0157297887 | Train Accuracy = 99.40%

Appendix D: Webpage Design and Page Components

1. Service Architecture



[Figure 18] 사용자 사진 입력 이전 화면

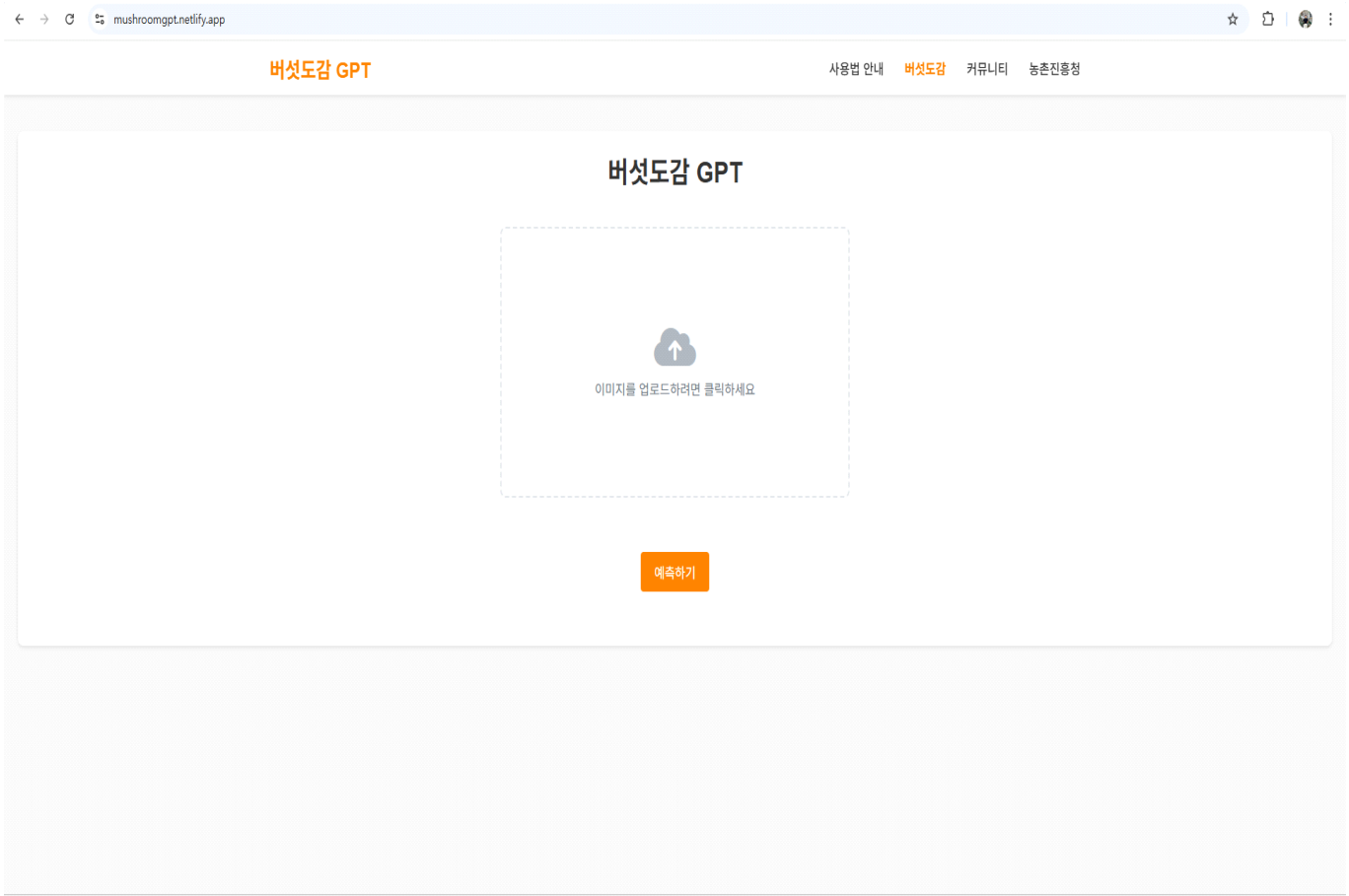
[Figure 18]은 구현한 본 서비스에서 제공하는 페이지의 구조를 간단하게 나타내고 있는 그림이다. 페이지별로 간략하게 소개하면 다음과 같은 기능을 하는 페이지로 이루어져 있다. 기본적으로 웹 페이지에 접속하면 index.html 페이지에 접속하게 되는데, 이는 사용자가 판단이 필요한 버섯의 사진을 입력하면, 버섯을 판단하여 기본적인 정보를 제공하는 페이지이다. notice.html은 제공하는 서비스를 어떻게 사용하는지, 어떻게 활용할 수 있는지를 간략하게 소개하는 페이지이다. 다음으로는 rda.html 페이지가 있다. 본 연구에서 제시한 문제에서 소개한 것처럼 독버섯에 관한 정보를 제공하고, 위험성을 알리는 정부 기관인 농촌진흥청의 홈페이지로 이동할 수 있도록 페이지를 구성하였다.

마지막으로는 본 서비스를 통해 기대할 수 있는 효과 중 하나로, 전문가와 일반인 간의 활발한 교류가 이어질 수 있는 페이지를 만들고자 하였다. 이를 통하여 현재 인터넷상에는 거의 없는 버섯과 독버섯에 관한 정보가 많이 생성되고, 이러한 정보가 공유되는 하나의 매개체가 되었으면 하여 community.html 페이지를 구현하였다. 세부 페이지로 글을 작성할 수 있는 write.html과 작성된 글을 볼 수 있는 post.html로 구성되어 있다. 이는 데이터베이스에 저장하여 관리하여야 하지만, 현재는 서비스 초기 단계이기에 데이터베이스 구축을 생략하였다. 페이지별로 구현한 세부적인 사항은 이어지는 장에서 서술하고자 한다.

2. Index.html(Main Page)

Index 페이지는 제공하는 서비스의 가장 메인이 되는 페이지로, 사용자가 판단을 원하는 버섯의 사진을 올릴 수 있도록 하며, 이후 해당 버섯이 독버섯인지에 관한 여부와 버섯의 정보를 간략하게나마 얻을 수 있도록 제공하는 페이지이다. 해당 페이지의 세부 구성은 다음과 같다.

2.1. 사용자 사진 입력 이전 페이지



[Figure 19] 사용자 사진 입력 이전 화면

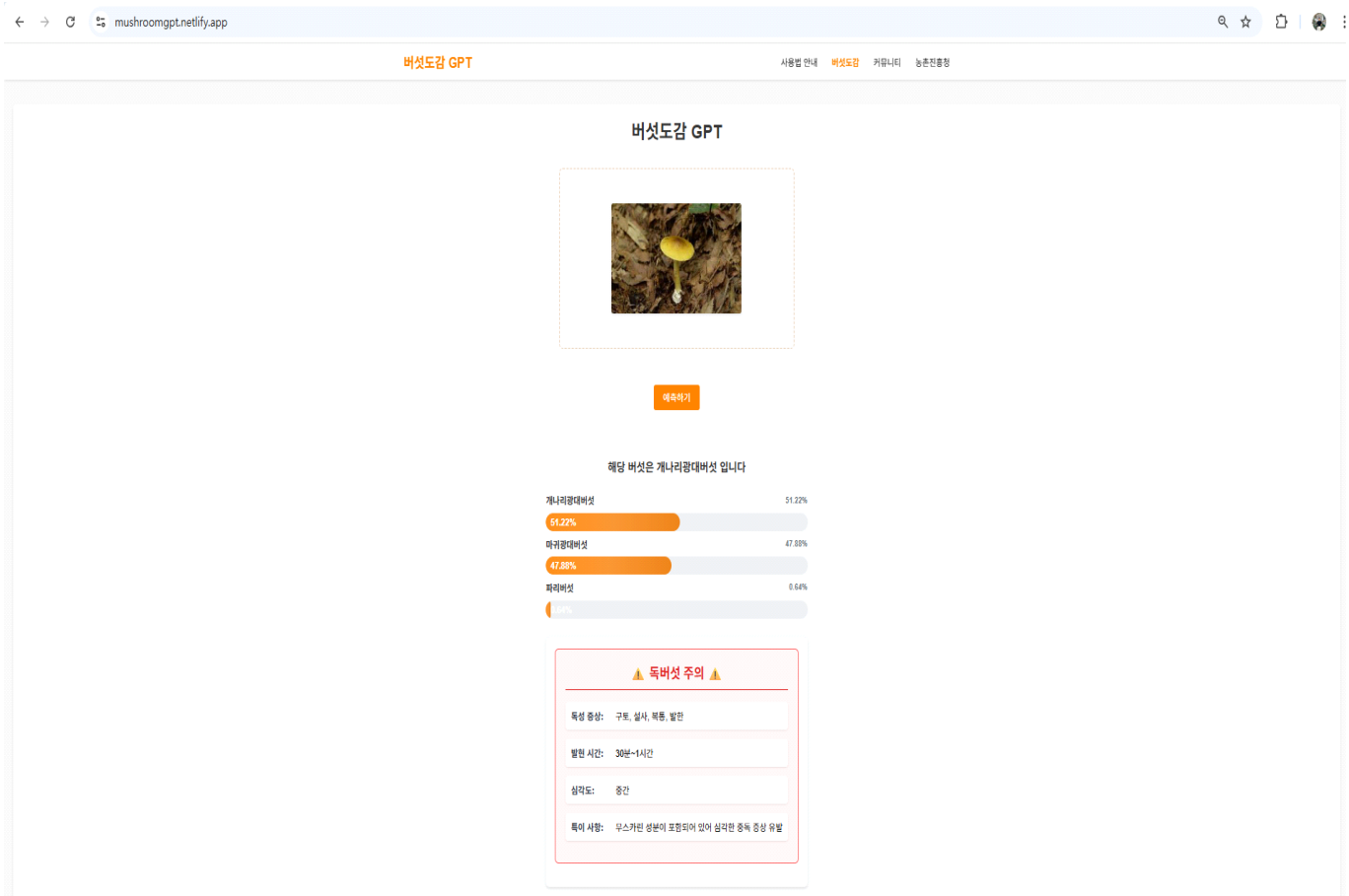
[Figure 19]는 사용자가 사진을 입력하기 이전, 페이지에 접속하면 나타나는 기본적인 주 페이지의 화면이다. 해당 페이지의 자세한 구현은 다음과 같다.

맨 위는 모든 페이지의 공통적인 페이지로, Navigation Bar로 구현하여 다른 페이지로 이동할 수 있도록 서비스를 제공한다. 왼쪽 주황색 글씨로 된 '버섯도감 GPT'를 클릭하면 지금의 주 페이지로 이동하며, 오른쪽에 있는 '사용법 안내', '커뮤니티', '농촌진흥청'을 선택하면, 앞으로 소개할 다른 페이지로 이동하게 된다. '버섯도감' 페이지는 현재의 페이지를 가리키고 있어 현재의 주 페이지로 이동한다.

가운데에는 사용자가 이미지를 올릴 수 있도록 기능을 제공하고 있다. 설정해 놓은 구역으로 마우스를 올리거나 클릭하면 이미지를 선택하여 올릴 수 있도록 구현하였으며, 해당 이미지는 모델에 입력되는 이미지가 된다.

'예측하기' 버튼은 사용자가 이미지를 올렸을 경우, 해당 버튼을 클릭하게 되면 이미 학습한 모델이 해당 이미지를 보고 어떠한 버섯인지를 판단하여 사용자에게 다시 반환할 수 있도록 Back-End에 이벤트를 전달하는 역할을 한다. 해당 버튼 없이 입력을 기다리다가 이미지가 올라오면 곧바로 수행하도록 구현할 수도 있겠지만, 오류 발생을 최소화하기 위하여 버튼 클릭을 하게 되면 이후 과정을 수행할 수 있도록 해당 방법을 선택하였다.

2.2. 사용자 사진 입력 이후 페이지

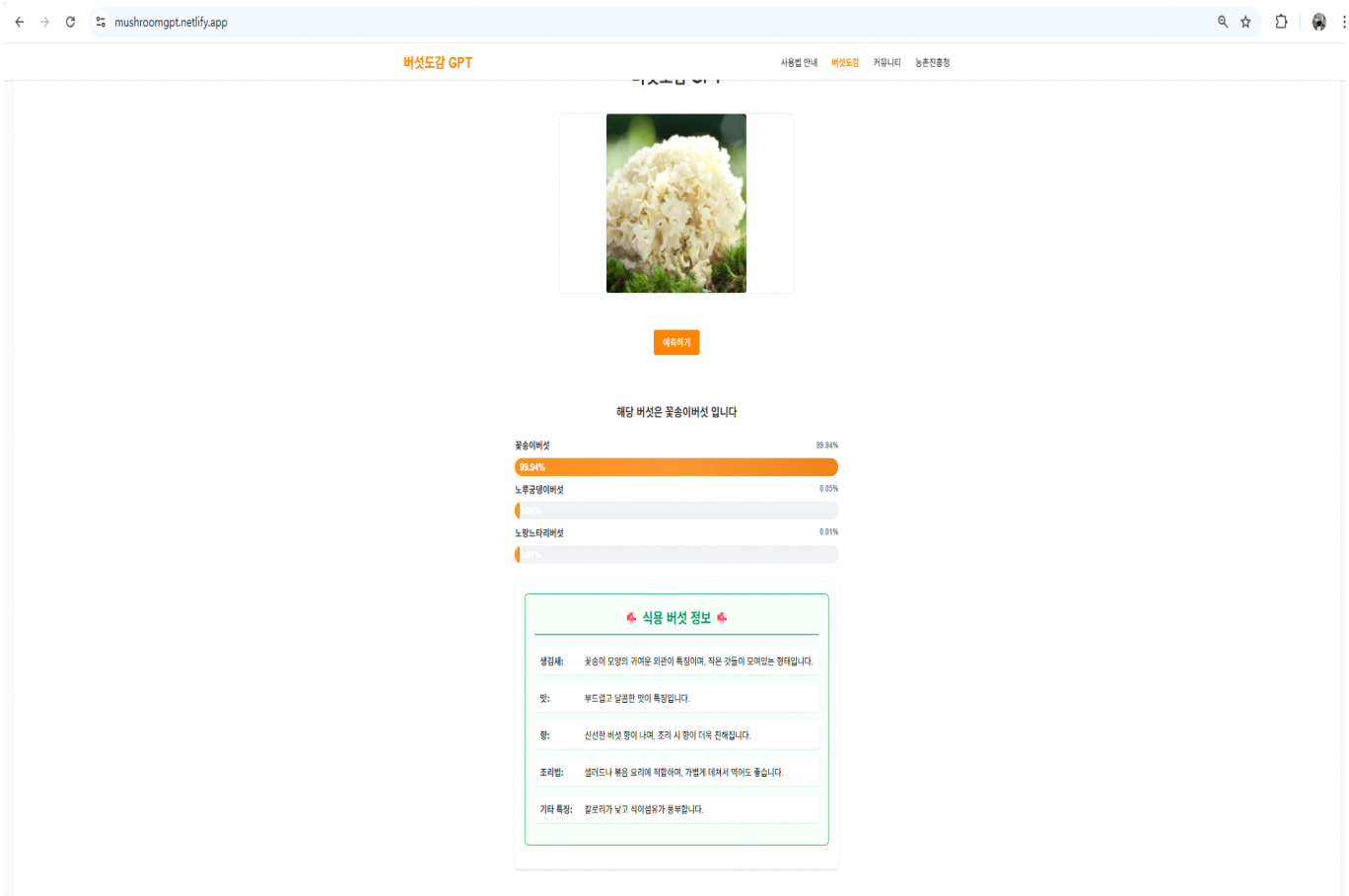


[Figure 20] 사용자 사진 입력 이후 화면(1)

[Figure 20]은 사용자가 올린 버섯의 이미지를 판단한 결과, 해당 버섯을 독버섯이라고 판단했을 때 사용자에게 보이는 화면이다. '예측하기' 버튼 아래에 제공하고 있는 기능을 설명하면 다음과 같다.

먼저, 해당 버섯이 어떠한 버섯인지를 설명한다. 해당 버섯의 이름은 본문에서 설명한 바와 같이 모델은 버섯별로 확률을 최종적으로 나타내게 되는데, 가장 높은 확률을 보이는 버섯의 이름을 판단한 버섯으로 이해하고, 해당 버섯의 이름을 보여준다. 그 아래에는 해당 버섯이 어느 정도로 판단했는지를 간략하게 함께 보여준다. 해당 예시의 경우 개나리광대버섯으로 판단한 확률이 51.22%이고, 마귀광대버섯이 47.88%, 파리버섯이 0.64%임을 나타내고 있다. 즉, 해당 사진을 보았을 때 51% 정도의 신뢰도로 개나리광대버섯이라 판단하였음을 설명하고자 다음과 같은 정보 형태로 제공한다. 더 나아가서 애니메이션을 통해 확률을 보여주는 그래프가 올라감을 시각적으로 추가로 표현하였다.

이후, 아래에는 독버섯임을 나타내도록 빨간색으로 경고성을 부각했으며, 안의 내용은 독성 증상, 독성의 발현 시간, 심각도, 특이 사항을 담아 사용자에게 제공한다. 이를 통해서 판단한 버섯을 인터넷상에 다시 검색하여 정보를 찾는 불편함을 조금이나마 감소시키고자 해당 정보를 함께 제공하고자 구현하였다.



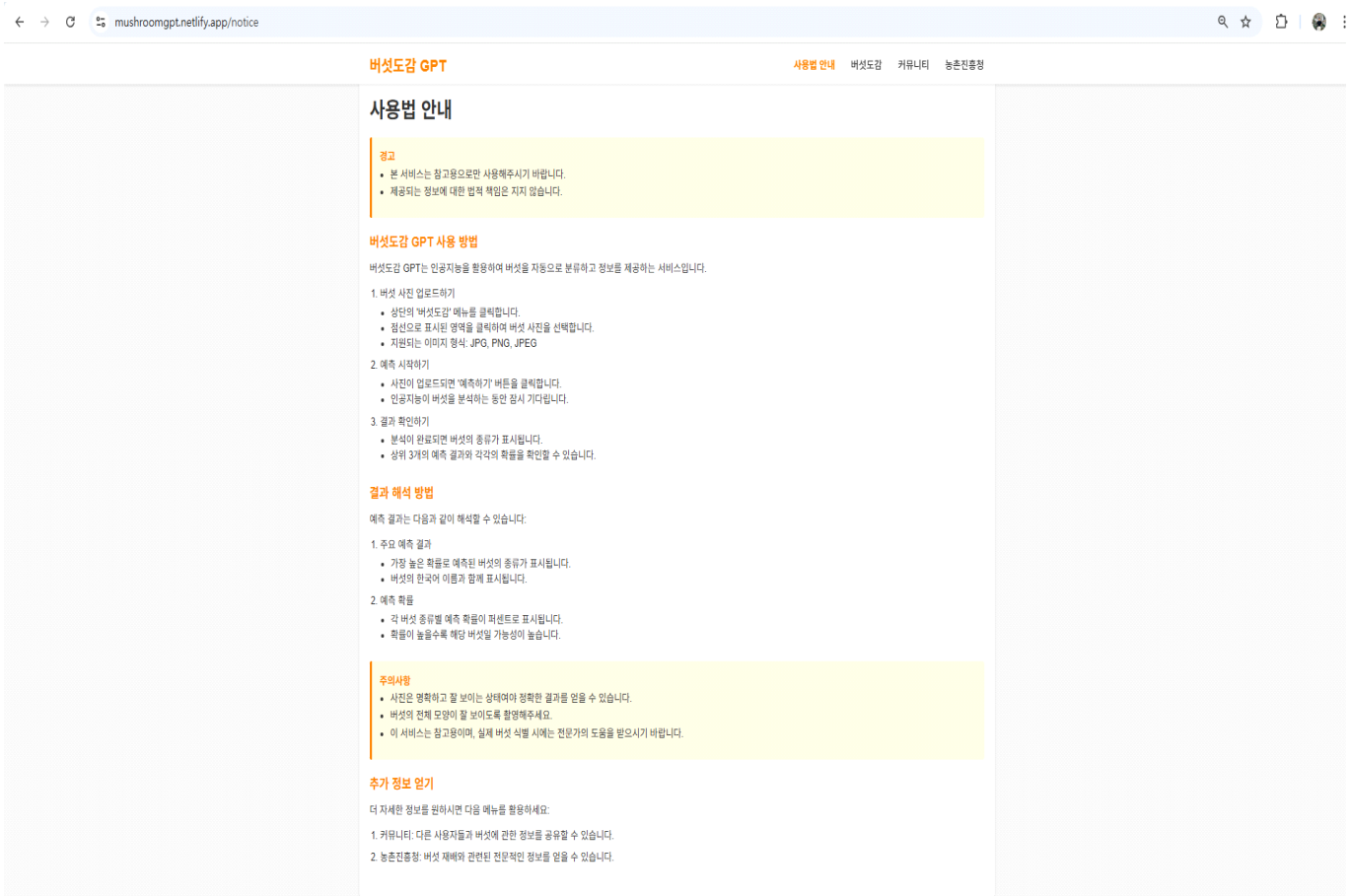
[Figure 21] 사용자 사진 입력 이후 화면(2)

[Figure 21]은 [Figure 20]과는 다르게, 사용자가 올린 버섯의 이미지를 판단한 결과, 해당 버섯을 식용버섯이라고 판단했을 때 사용자에게 보이는 화면이다.

식용버섯일 때도 어느 정도의 확률로 판단하였는지를 나타내고 있으며, 식용버섯은 독버섯과는 다르게 안전함의 의미를 담아 초록색으로 배경색을 두었다. 또한, 독버섯일 때 제공하는 정보처럼 독성의 정보나 발현 시간, 심각도를 제공할 필요가 없으므로, 식용버섯일 때는 생김새와 맛, 향과 조리법, 그리고 기타 특징을 간략하게 요약하여 사용자에게 제공한다.

3. Notice.html

Notice 페이지는 해당 서비스를 어떻게 사용해야 하는지에 관한 정보를 제공하는 사용법 안내 페이지이다.



[Figure 22] 사용법 안내 화면

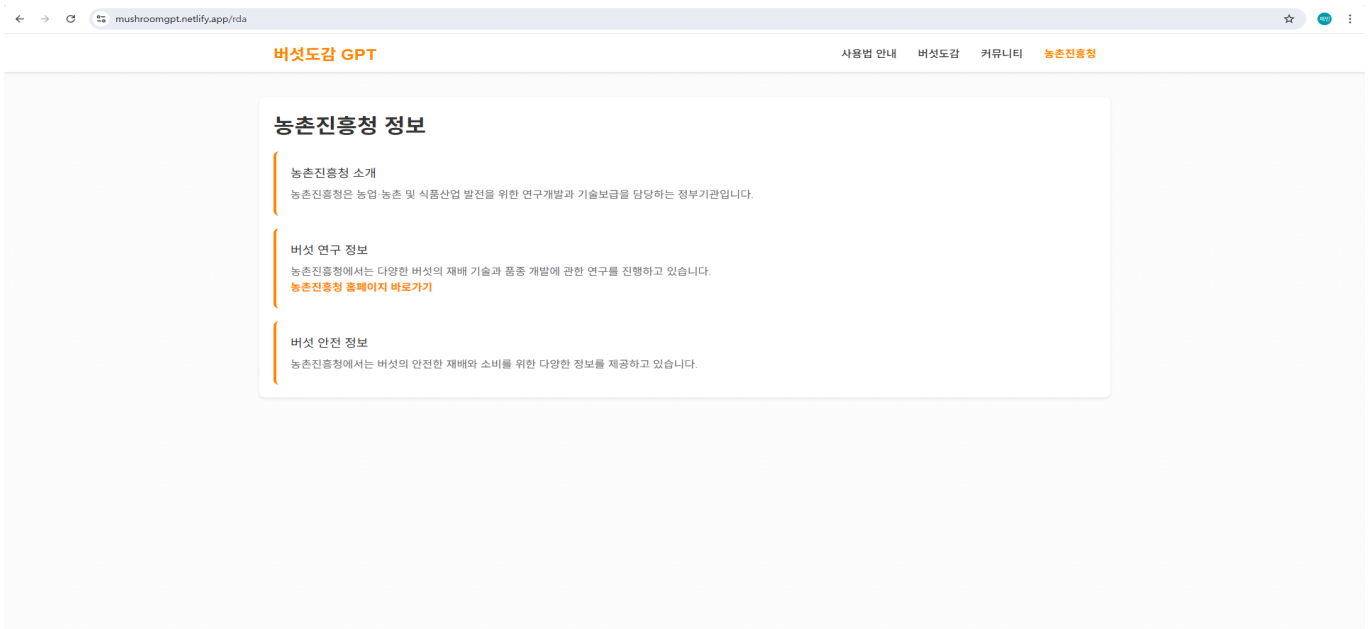
맨 처음에는 해당 페이지는 참고용으로만 사용할 수 있도록 안내함으로써, 해당 서비스를 맹목적으로 믿지 않도록 방지하고 있다. 이후에는 제공하는 메인 서비스(버섯 판단하기)를 사용하기 위해서 어떻게 해야 하는지를 설명한다. 주 페이지에 다른 기능을 제공하지 않기 때문에 접근하기에는 쉽다고 생각하지만, 어려움을 겪을 수 있는 사용자를 위하여 사용 방법에 대해 글로써 순서대로 제공하고 있다.

이후에는 결과를 해석하는 방법을 제공하고 있다. 가장 높은 확률로 예측된 버섯의 종류가 표시된다는 문구를 통하여 가장 높은 확률을 보인 버섯이 판단된 버섯이라고 이해할 수 있도록 하였으며, 주의 사항으로 마지막에 한 번 더 해당 서비스는 참고용이기에 정확한 판단은 전문가의 도움을 받아야 함을 상기시키고자 하였다.

마지막에는 본 서비스에서 제공하는 커뮤니티, 농촌진흥청 페이지를 간략하게 소개하고 있다. 이를 통해서 다른 페이지에서 제공하는 서비스는 어떠한 목적으로 제공되고 있는지를 이해할 수 있도록 추가하였다.

4. rda.html

RDA 페이지는 각 버섯의 정보를 더욱 자세하게 얻을 수 있는 농촌진흥청으로 연결되도록 하는 페이지이다.

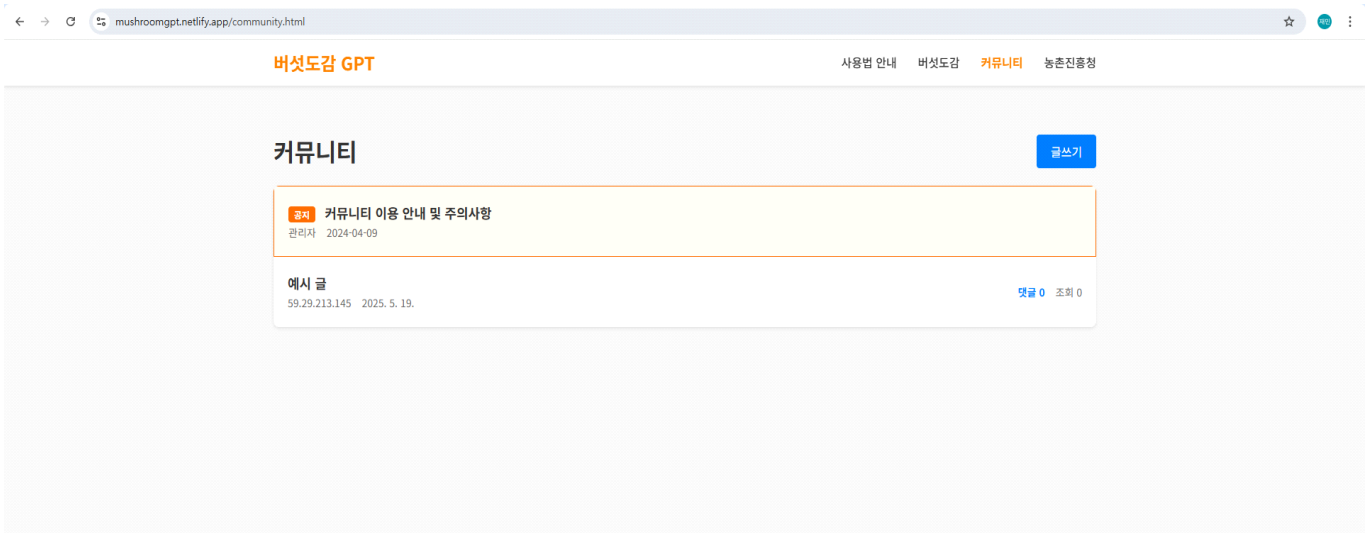


[Figure 23] 농촌진흥청 소개 화면

[Figure 23]은 농촌진흥청을 클릭하였을 때 보이는 페이지이다. 해당 페이지에서는 농촌진흥청이 어떠한 역할을 담당하는지, 그리고 버섯의 정보를 더욱 얻기 위해서는 어느 경로로 이동해야 하는지를 사용자에게 전달하는 것을 목적으로 둔다. 다른 기관들도 존재하지만, 매년 가을철이 되면 독버섯에 관련된 잘못된 정보를 바로잡을 수 있는 정보와 독버섯을 주의해야 한다는 내용을 담은 다양한 정보들을 정리하여 제공하는 정부 대표 기관이 농촌진흥청이기 때문에, 농촌진흥청으로 이동할 수 있도록 구현하였다.

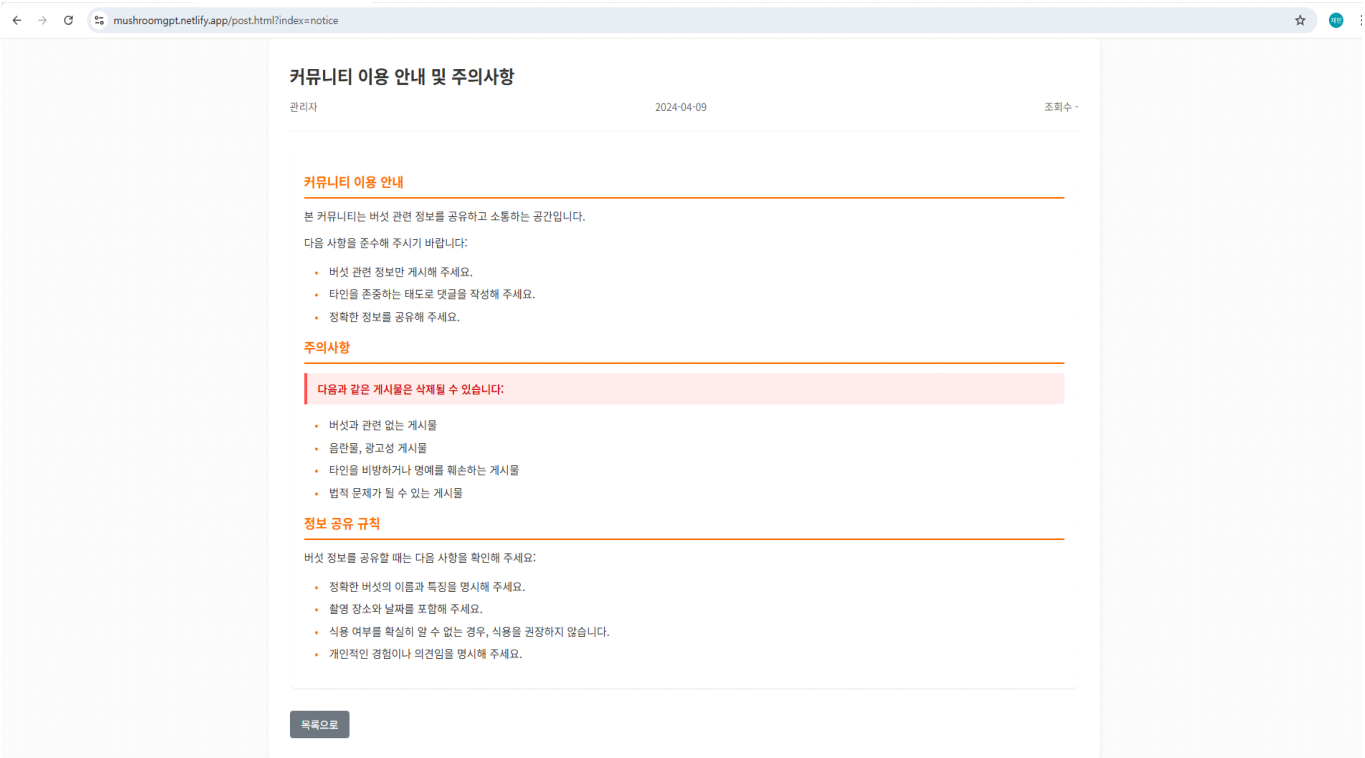
5. community.html

Community 페이지는 온라인상에서 버섯에 관한 정보를 얻기 힘들다는 문제점을 해결할 수 있도록 하는 하나의 매개체가 되었으면 하는 목적을 가지고 구현하였다.



[Figure 24] 커뮤니티 주 페이지 화면

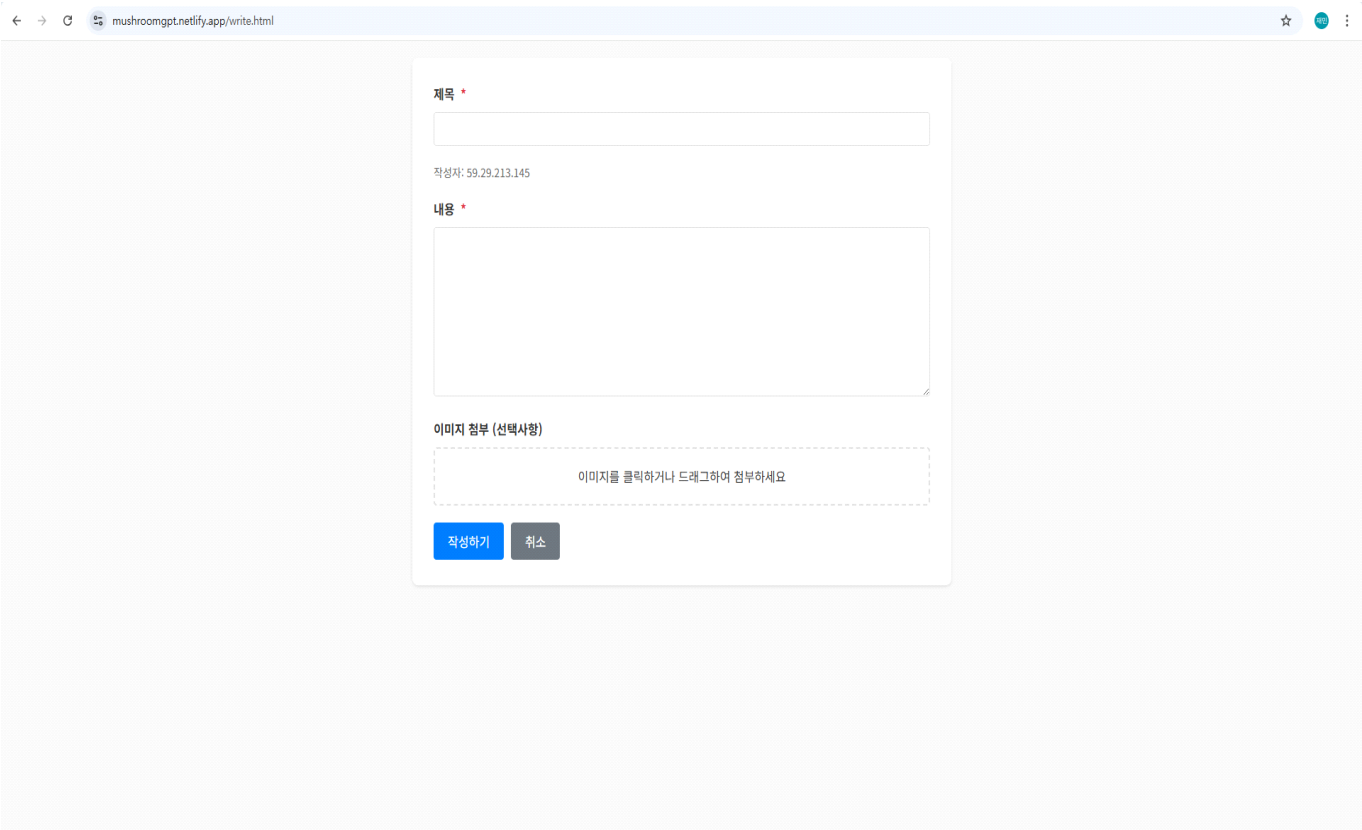
[Figure 24]는 커뮤니티를 클릭하였을 때 보이는 페이지이다. 이 페이지에서는 작성된 글을 확인할 수 있도록 제공하며, 오른쪽 위에 있는 글쓰기 버튼을 통하여 커뮤니티에 글을 작성하여 남길 수 있도록 기능을 제공하고 있다. 가운데에는 현재 작성된 글을 확인할 수 있는데, 맨 위에는 커뮤니티 이용 안내 및 주의 사항을 사용자가 알 수 있도록 맨 위에 고정하였다.



[Figure 25] 커뮤니티 이용 안내 및 주의 사항 안내 화면

커뮤니티 이용 안내와 주의 사항을 나타내고 있는 페이지를 접속하면 [Figure 25]와 같은 화면을 확인할 수 있다. 커뮤니티 이용 안내로는 버섯에 관련된 정보를 공유하고 소통하는 공간임을 밝히면서, 해당 목적의 글이 작성될 수 있도록 안내하고 있다. 주의 사항으로는 문제가 발생할 수 있는 게시물의 종류와 해당 글은 통보 없이 삭제될 수 있음을 명확하게 밝힘으로써 다시 한번 더 해당 서비스의 목적을 상기시키는 역할을 한다. 마지막으로는 규칙을 설명하며 사용자가 커뮤니티 서비스를 통하여 버섯에 관한 정보를 자유롭게 공유하고 작성할 수 있도록 하였다.

5.1. write.html



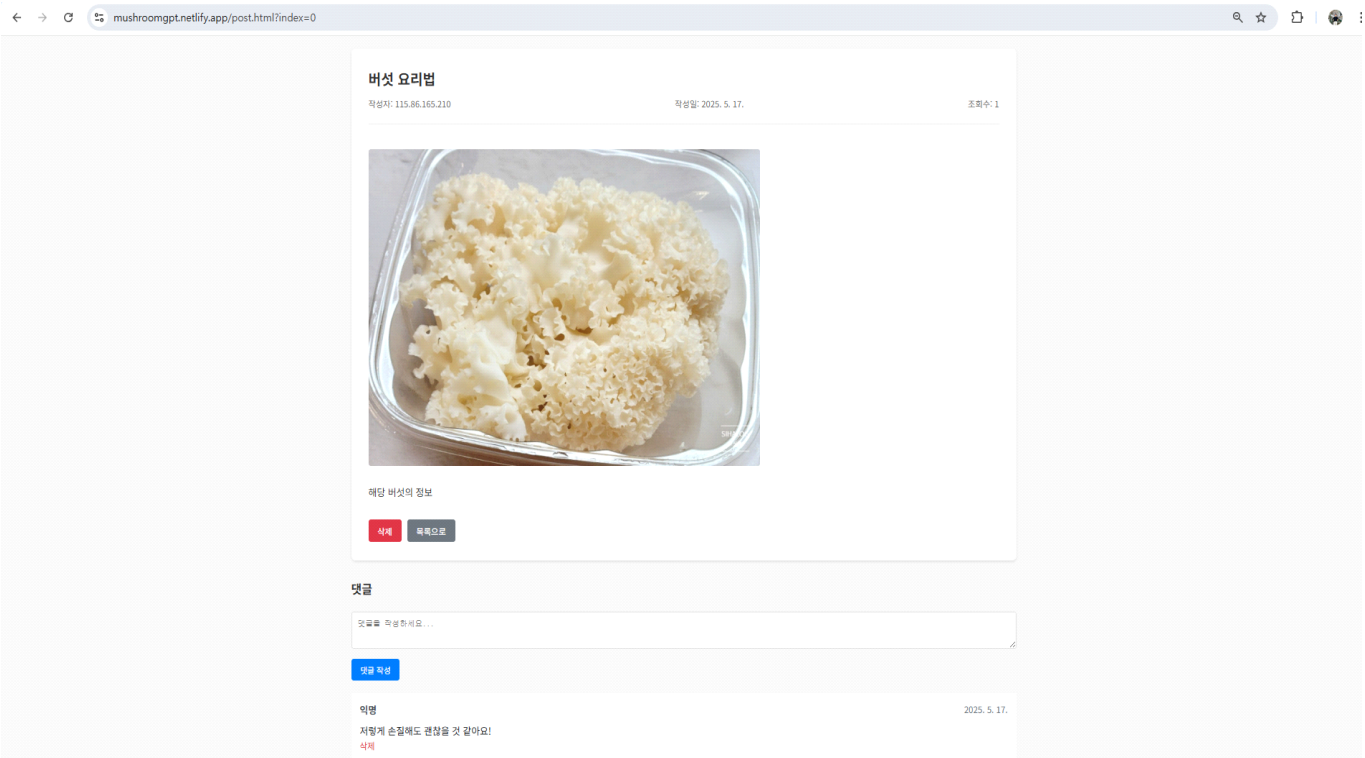
[Figure 26] 글쓰기 화면

[Figure 26]은 [Figure 24] 페이지 내 오른쪽 위에 있는 글쓰기 버튼을 눌렀을 때, 커뮤니티에 글을 남길 수 있도록 제공하는 페이지이다. 제목을 입력할 수 있도록 하였으며, 그 아래에는 작성자의 IP 정보를 나타내게 하였다. 작성자의 정보로 별명이나 이름을 작성토록 할 수도 있지만, 사용자의 IP 주소를 나타내게 함으로써, 이후 법적인 문제가 발생하였을 때 해당 정보를 기반으로 추적할 수 있음을 암시적으로 나타내고자 하였다. 해당 기능을 통하여 커뮤니티의 규칙을 지키고, 해당 서비스의 목적을 해치지 않도록 하는 효과를 기대할 수 있다.

다음으로는 내용을 입력할 수 있도록 하며, 마지막으로는 선택사항으로 이미지를 첨부할 수 있도록 하였다. 이미지 첨부는 버섯의 정보가 궁금하거나, 메인 서비스에서 판단한 값이 모호할 때 전문가나 다른 사용자의 의견을 얻을 때 수월하도록 해당 기능을 제공한다. 이후, 작성하기 버튼을 누르게 되면 [Figure 24]의 중간 부분에 있는 '예시 글'의 제목을 가지고 있는 부분처럼 글이 생성되고, 다른 사용자가 확인할 수 있게 된다.

5.2. post.html

Community 페이지에서 다른 사용자가 작성한 글을 클릭하게 되면, 해당 글 내의 정보를 확인할 수 있도록 Post 페이지를 제공한다.



[Figure 27] 글 내용 확인 화면

[Figure 27]의 형태로 사용자에게 이전에 작성된 글을 확인할 수 있도록 제공하고 있으며, Post 페이지는 이전에 작성하여 올려둔 글을 예시로 설명하고자 한다. 먼저 글의 제목과 작성자의 IP 주소, 작성일은 언제인지 확인할 수 있도록 하였다. 다음으로는 작성자가 선택사항으로 첨부한 이미지 사진이 보이게 되며, 그 아래에는 [Figure 26]에서 내용에 해당하는 정보가 나타나도록 하였다. 여기까지는 [Figure 26]에서 나타나는 구성인 제목, 내용, 이미지를 확인할 수 있다.

Post에서 제공하는 또 하나의 기능으로 댓글 기능을 제공하고 있다. 단순히 글을 게시하는 것만으로는 사용자 간 교류에 한계점이 명확하다. 만약, 어떤 하나의 글이 질문을 통해 의견을 얻고자 하는 글이라 가정하였을 때, 댓글 기능이 없다면 새롭게 글을 작성하여 게시함으로써 소통이 이루어지도록 해야 하기에, 댓글 기능을 제공함으로써 이러한 제약사항이 없도록 서비스를 구현하였다. 댓글의 수와 조회 수는 [Figure 24]에서 글의 오른쪽 지점에서 확인할 수 있다.

6. Database

커뮤니티 기능이 원활하게 작동하려면 사용자가 게시한 글이 다른 사용자에게 보일 수 있게 하도록 데이터베이스를 구축하여, 하나의 공간에 저장되도록 하여야 한다. 하지만, 지금 상황에서는 딥러닝을 이용한 모델을 간단하게 구현하고, 해당 모델을 활용하여 간단한 서비스를 제공하는 것을 목적으로 두기에, 데이터베이스는 구현하지 않았다.

만약, 해당 서비스를 모두가 사용할 수 있도록 배포하고자 하면 데이터베이스를 구축하여 글이 저장되어 모두에게 공유될 수 있도록 하여야 한다. 이 외에도 미비한 부분과 법적인 틀 안에서 올바른 정보를 제공할 수 있도록 수정이 필요하다.

Appendix E: Team Member Introduction, Roles, and Contributions

황 재 민 (Jaemin Hwang)

Email: codesensory@gmail.com

2025년 ~ 현재: 강릉원주대학교
컴퓨터공학과 학부생

Roles

- 학습 이미지 수집, 증강기법 적용
- 모델 구현 및 학습
- 웹 페이지 구현 및 구축
- 프로젝트 보고서 작성
- 발표 자료 제작, 수정 및 보완
- 최종발표 담당

전 준 현 (Junhyeon Jeon)

Email: wjsgus1549@naver.com

2022년 ~ 현재: 강릉원주대학교
컴퓨터공학과 학부생

Roles

- 학습 이미지 수집
- 발표 자료 수정 및 보완
- 웹 페이지 구현
- 발표 자료 수정 및 보완

이 창 규 (Changgyu Lee)

Email: unisusu@gwnu.ac.kr

2025년 ~ 현재: 강릉원주대학교
컴퓨터공학과 학부생

Roles

- 학습 이미지 수집
- 발표 자료 제작 및 수정
- 중간발표 담당
- 주간 연구일지 취합

지 수 정 (Sujeong Ji)

Email: jisujeong670412@gmail.com

2022년 ~ 현재: 강릉원주대학교
컴퓨터공학과 학부생

Roles

- 학습 이미지 수집
- 발표 자료 제작 및 수정
- 프로젝트 보고서 오·탈자 확인

	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
이전 연구, 서비스 분석														
Data-set 구성														
CNN 모델 구현														
모델 학습, Fine-Tuning														
학습 결과 분석														
웹 페이지 서비스 구현														
연구 보고서 작성														
발표 자료 작성, 보완														

[Table 6] Project Schedule